



TheDBAdmin

Practitioner Field Guide Series

Modern Linux Commands for the Modern DBA

A Practical Migration Guide from Legacy Tools to Modern Replacements

50 Command Pairs · Real-World DBA Scenarios · Production-Tested Patterns

Versha

Founder, TheDBAdmin.com
Agentic AI Corporate Trainer | Database Administration

Edition 1.0 · 2026

Modern Linux Commands for the Modern DBA

A Practical Migration Guide from Legacy Tools to Modern Replacements

© 2026 TheDBAdmin.com. All rights reserved.

First Edition: 2026

Published by TheDBAdmin.com

Notice of Rights

No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means electronic, mechanical, photocopying, recording, or otherwise without the prior written permission of the publisher, except for brief quotations embodied in critical reviews and certain other non-commercial uses permitted by copyright law. For permission requests, write to TheDBAdmin.com.

Notice of Liability

The information in this book is distributed on an “As Is” basis, without warranty. While every precaution has been taken in the preparation of this book, neither the author nor TheDBAdmin.com shall have any liability to any person or entity with respect to any loss or damage caused or alleged to be caused directly or indirectly by the instructions or example commands contained herein. Always validate commands in a non-production environment before applying them to production database systems.

Trademarks

Linux is the registered trademark of Linus Torvalds. PostgreSQL is a trademark of the PostgreSQL Global Development Group. MySQL is a trademark of Oracle Corporation. systemd is a trademark of Red Hat, Inc. All other product names, logos, and brands referenced are the property of their respective owners. Use of these names does not imply endorsement.

Source Verification

Every command documented in this book has been cross-checked against official manual pages and primary project documentation. Where commands have known portability constraints, those constraints are stated explicitly. Where modern tools are forks of deprecated upstream projects, the maintenance status is noted (for example, eza as the maintained fork of the deprecated exa).

Contact: TheDBAdmin.com | LinkedIn: /in/thedbadmin | Author: Versha

Preface

Database administrators occupy a peculiar position in modern infrastructure. We are simultaneously the custodians of the most valuable, most regulated, and least forgiving subsystem in the stack — the database — and the users least likely to upgrade our personal tooling. The reason is rational: a DBA who learned `ifconfig`, `netstat`, and `route` in 2008 has spent fifteen years building muscle memory and dashboards around those commands, and there is no incident report that thanks anyone for typing a faster `ls`.

Yet the ground has shifted beneath us. The `net-tools` package containing `ifconfig` has not received a meaningful update in over a decade and has been marked deprecated on every major distribution. Containers ship without it. Cloud images strip it out. The first time a junior engineer pages you at 02:30 because the database host's "ifconfig is broken," you discover that the host simply does not ship `ifconfig` anymore, and your runbook is now a liability.

This book is the migration guide I wish I had been handed in 2018. It pairs fifty traditional Linux commands with their modern replacements, organized by the order a DBA actually encounters them during an on-call shift: networking first, then services, then processes, then files, then transfer and compression, then logging and scheduling. For each pair the book follows the same pedagogical structure: the problem the original command was designed to solve, the gap that emerged as workloads modernized, the replacement and what it does differently, a real-world DBA scenario where the difference matters, the exceptions where the legacy command is still the right answer, comparable alternatives, and a short conclusion.

The audience is the practicing DBA — Oracle, PostgreSQL, MySQL, MongoDB, SQL Server, or any combination — who already runs production Linux every day. The book does not teach Linux from scratch. It assumes you understand processes, file descriptors, sockets, and the difference between TCP and UDP. It does not assume you have ever heard of `ss`, `journalctl`, `eza`, `ripgrep`, or `zstd`.

Two warnings before you continue. First, never run an unfamiliar command for the first time on a production database host. Every example in this book has a non-destructive form; use it. Second, the modern alternatives in this book solve real problems, but they are not a religion. A book that tells you to replace every legacy command in your runbook on Monday morning is selling you a regression, not a migration. Treat this as a menu, not a mandate.

Versha

Founder, TheDBAdmin.com

Bengaluru, 2026

Table of Contents

This volume is organized in seven parts, twenty-five chapters, and two appendices. Page numbers below correspond to the rendered document; right-click the entry in Word and choose Update Field to re-flow page numbers if the document is edited.

Preface	4
Part I · Why Legacy Commands No Longer Suffice	6
Chapter 1 — Why Legacy Commands No Longer Suffice	6
Part II · Networking.....	8
Chapter 2 — From ifconfig to ip.....	9
Chapter 3 — From route to ip route	12
Chapter 4 — From netstat to ss	14
Chapter 5 — ARP, Bridges, and Wireless	17
Chapter 6 — DNS Tools: From nslookup to dig.....	20
Chapter 7 — Connectivity Testing: ping, traceroute, mtr.....	22
Chapter 8 — TCP Port Connectivity: From telnet to nc	24
Chapter 9 — Secure File Transfer: From ftp to sftp	26
Part III · System and Service Management	28
Chapter 10 — From SysV init to systemd.....	29
Chapter 11 — Logging: From /var/log to journalctl.....	32
Chapter 12 — Scheduling: From cron to systemd Timers	35
Part IV · Process and Resource Monitoring	38
Chapter 13 — Process Inspection: ps, pgrep, top, htop, btop.....	39
Chapter 14 — Memory and Disk: free, duf, dust	41
Part V · File Operations and Text Processing	43
Chapter 15 — File Listing: From ls to eza	44
Chapter 16 — File Viewing: From cat / less to bat.....	46
Chapter 17 — Finding Files: From find to fd	48
Chapter 18 — Content Search: From grep to ripgrep.....	50
Chapter 19 — Text Manipulation: sed → sd, awk → mlr.....	52
Part VI · HTTP, Transfer, and Compression	55
Chapter 20 — curl → HTTPie.....	56
Chapter 21 — wget → curl -O / aria2c.....	58

Chapter 22 — scp → rsync	60
Chapter 23 — gzip / tar -czf → zstd / tar --zstd	63
Part VII · Migration Strategy.....	66
Chapter 24 — Adoption Strategy for DBA Teams	67
Chapter 25 — Building a Modern DBA Toolkit.....	70
Appendix A · Quick Reference Cheat Sheet.....	73
Appendix B · Installation Guide.....	76
References.....	79
About TheDBAdmin and the Author	81

Chapter 1 — Why Legacy Commands No Longer Suffice

1.1 The Problem

The Linux commands a DBA learned between 1995 and 2010 were designed for a server that no longer exists. That server had one or two network interfaces, a static IP, a single root filesystem on spinning disk, no namespaces, no cgroups, no systemd, and a kernel that fit comfortably in the RAM of the workstation under the operator's desk. The net-tools package — `ifconfig`, `route`, `netstat`, `arp` — was a perfect fit for that server because it spoke the language of that server.

Modern database hosts are very different. A single Kubernetes worker node running a PostgreSQL StatefulSet may have a dozen virtual interfaces, three routing tables, a service mesh sidecar, cgroup-scoped memory accounting, and bonded NICs over VLANs. Cloud database instances expose CNI plugins, ENIs, and SR-IOV adapters whose configuration `ifconfig` literally cannot describe. The mismatch is not aesthetic; it is structural.

Three converging pressures forced the modernization wave:

- **Maintenance attrition.** The net-tools package has not received a substantive release in years; security fixes have stalled. Distributions including Arch, RHEL 8 and later, and minimal Debian and Ubuntu images have stopped installing it by default.
- **Feature gap.** Network namespaces, VRF, eBPF socket diagnostics, and IPv6 dual-stack semantics simply do not exist in the net-tools mental model. Tools written against that model cannot represent the underlying kernel state correctly.
- **Container reality.** A diagnostic command that depends on `/proc` parsing of textual tables breaks the first time the host's network is virtualized. The replacement tools query the kernel through netlink and produce machine-parseable output that survives the transition into containers.

1.2 What This Book Does Differently

This book is not a generic cheat sheet. The internet has dozens of those, and they all have the same defect: they show two commands side by side without telling you when the substitution is safe. The structure used in every chapter of this book is deliberate and follows the teaching order a DBA needs to actually trust a new command in production:

Pedagogical Structure (used in every command pair)

1. The Problem — what the original command was designed for and what shifted.
2. The Modern Solution — what replaced it and what it does that the original cannot.
3. DBA Real-World Scenario — a concrete production situation where the difference is operationally meaningful.
4. When the Legacy Command Still Wins — the exceptions, edge cases, and portability traps.

5. Comparable Tools — adjacent alternatives worth knowing.
6. Conclusion — a one-sentence recommendation.

1.3 How to Use This Book

Read Chapter 2 to calibrate your mental model of the modernization wave. After that, the book is reference material. The Quick Reference Cheat Sheet in Appendix A reproduces all fifty pairs in one table, sorted by the operational domain (networking, services, processes, files, transfer, compression, logging, scheduling) so that the chapter you need is always at most one table lookup away.

Every code example in this book has been tested on a current Ubuntu LTS release and verified against the upstream manual page. When a command behaves differently on RHEL, Arch, Alpine, or macOS, that difference is called out explicitly in the chapter.

1.4 A Word on Risk

Production Safety

Never run an unfamiliar command for the first time on a production database host.

Every example in this book has a non-destructive form (`--dry-run`, `-n`, `--no-act`, or read-only flags). Use it.

Validate the output format before piping it to a script. `ss` and `ip` output are NOT drop-in replacements for `netstat` and `ifconfig` output in your existing parsing logic.



Part II — Networking

Networking is where the modernization wave is most visible and most operationally consequential for DBAs. The chapters in this part cover ten command pairs that together account for nearly every networking question a DBA asks: which interfaces does this host have, what address are they on, where do my packets go, what is listening on a port, what is the ARP table saying, can I reach the upstream, and what name does this DNS server resolve. The replacement story is unusually clean here: a single tool, `ip`, from the `iproute2` package replaces `ifconfig`, `route`, and `arp`; a single tool, `ss`, replaces `netstat` for socket diagnostics; and a single tool, `dig`, replaces `nslookup` and `host` for DNS troubleshooting.

Why DBAs care about networking commands more than most sysadmins

A database is, at its core, a network service. Every connection refused, every replication lag spike, every slow query that turns out to be a slow TCP retransmit, lands on the DBA's pager. The faster you can read the kernel's network state, the faster you close the ticket.

Chapter 2 — From ifconfig to ip

2.1 The Problem

ifconfig (interface configuration) first shipped with 4.2BSD in 1983 and was ported into Linux's net-tools package shortly afterward. For two decades it was the canonical way to ask a Linux host “which network interfaces do you have and what addresses are on them?” Its mental model was simple: an interface had a name (eth0, eth1), a primary IPv4 address, a netmask, a broadcast address, and an up/down flag. That model worked perfectly for a server with one or two physical NICs and static configuration.

The mismatch began when Linux acquired multiple IP addresses per interface (a feature ifconfig represented poorly using virtual aliases like eth0:0), then IPv6 (ifconfig's IPv6 output is incomplete on many implementations), then bonding and VLANs (ifconfig cannot create them), then network namespaces (ifconfig cannot enter them), then VRF, VXLAN, and macvlan. By 2016 the net-tools maintainers had effectively stopped feature work; major distributions began dropping the package from default installs.

2.2 The Modern Solution

The replacement is ip, part of the iproute2 package, which talks to the kernel through netlink rather than parsing /proc text files. Where ifconfig conflates layer-2 and layer-3 concerns into a single output, ip cleanly separates them: ip link manages the layer-2 device (state, MTU, MAC address), ip addr manages layer-3 addresses (IPv4 and IPv6 together), ip route manages the routing tables, and ip neigh manages the neighbor (ARP and IPv6 ND) cache. The structured separation is not academic; it is the reason ip can represent modern networking primitives at all.

Side-by-side syntax

Task	Legacy (ifconfig / net-tools)	Modern (ip / iproute2)
Show all interfaces and addresses	<code>ifconfig -a</code>	<code>ip addr show</code>
Show one interface	<code>ifconfig eth0</code>	<code>ip addr show eth0</code>
Bring an interface up	<code>ifconfig eth0 up</code>	<code>ip link set eth0 up</code>
Bring an interface down	<code>ifconfig eth0 down</code>	<code>ip link set eth0 down</code>
Add an IP address	<code>ifconfig eth0 10.0.0.5 netmask 255.255.255.0</code>	<code>ip addr add 10.0.0.5/24 dev eth0</code>
Remove an IP address	<code>ifconfig eth0:0 down</code>	<code>ip addr del 10.0.0.5/24 dev eth0</code>
Change MTU	<code>ifconfig eth0 mtu 9000</code>	<code>ip link set eth0 mtu 9000</code>

Task	Legacy (ifconfig / net-tools)	Modern (ip / iproute2)
Show interface statistics	ifconfig (counters at end)	ip -s link show eth0

2.3 DBA Real-World Scenario

Scenario: PostgreSQL streaming replica refuses to connect after a network change

A PostgreSQL streaming replica in your Mumbai data centre stops connecting to the primary in Hyderabad. The replica log shows connection refused. The replica host has been recently re-imaged as part of a fleet refresh; the operations team has migrated from a CentOS 7 image to RHEL 9, which no longer ships ifconfig by default. Your existing diagnostic runbook reads:

```
# Step 1: confirm interface is up
ifconfig eth0
# Step 2: confirm IP is correct
ifconfig eth0 | grep inet
```

Both lines now fail with “ifconfig: command not found.” The replacement diagnostic that works on every modern distribution is:

```
# Step 1: confirm interface is up and has the expected MTU
ip -br link show eth0

# Step 2: confirm the IPv4 address and look for any secondary address
ip -br addr show eth0

# Step 3 (the one ifconfig could not do): see what is actually on the wire
ip -s link show eth0

# Step 4: confirm a route to the primary's IP exists
ip route get 10.20.30.5
```

The -br (brief) flag is the practical upgrade. It produces one line per interface, ideal for grep and for pasting into a Slack incident channel without consuming twenty lines of screen real estate. The fourth command, ip route get, is the one a DBA discovers and immediately wonders how they ever lived without: it answers the question “if I send a packet to this IP right now, which interface and which gateway will the kernel use?” ifconfig cannot answer that question; route can only answer it indirectly by forcing you to compose the route table in your head.

2.4 When the Legacy Command Still Wins

- **BSD-derived systems.** On FreeBSD, OpenBSD, and macOS, ifconfig is not deprecated; it is the canonical and only tool. If your DBA fleet includes BSD-based jump hosts or macOS workstations, the muscle memory still pays.
- **Solaris (rare but extant).** Solaris uses ifconfig in conjunction with dladm. The iproute2 ip command is Linux-specific.

- **Decade-old runbooks.** If you are inheriting a CentOS 6 fleet that has not been upgraded, `ifconfig` is still installed and still works. Replace the runbook before the fleet, not after.

2.5 Comparable Tools

- **`nmcli`** (NetworkManager command-line interface): higher-level than `ip`; manages connection profiles persistently, integrates with `/etc/NetworkManager/system-connections/`. Use it when configuration must survive reboot on a NetworkManager-managed host.
- **`networkctl`** (`systemd-networkd`'s CLI): the equivalent for hosts using `systemd-networkd` instead of NetworkManager. Common on minimal cloud images and Kubernetes nodes.
- **`ethtool`** : purpose-built for layer-1 and driver-level diagnostics: link speed, duplex, autoneg state, NIC firmware version, ring buffer sizes, offload capabilities. A DBA debugging a slow replica should reach for `ethtool`, not `ip`.
- **`tcpdump`** : when `ip addr` says everything is fine but the connection still fails, `tcpdump` tells you what the wire actually carries.

2.6 Conclusion

Recommendation

Default to `ip` for all new scripts and runbooks. Keep `ifconfig` knowledge alive only for cross-platform compatibility (BSD, macOS) and for inherited legacy hosts. The brief output flag (`ip -br addr`) is the single biggest quality-of-life upgrade once you make the switch.

Chapter 3 — From route to ip route

3.1 The Problem

The `route` command (also part of `net-tools`) gave DBAs and sysadmins a flat, human-readable view of the kernel's main routing table. For a host with one default gateway and a handful of static routes, it was enough. But Linux routing has grown layered: there are multiple routing tables (the kernel maintains 255 of them, addressable by ID or name), policy rules that select tables based on source address, network namespace, or firewall mark, and equal-cost multi-path (ECMP) routes. `route -n` shows only table main, gives no policy rule context, and produces output that is awkward to parse.

More practically, `route` cannot add an IPv6 route, cannot create a route in a non-default table, cannot show which route the kernel would actually pick for a given destination, and treats ECMP as if it does not exist.

3.2 The Modern Solution

`ip route` is the routing-table half of the `iproute2` toolkit. It shows IPv4 and IPv6 with the same command (using `-4` or `-6` to filter), exposes the full multi-table model (`ip route show table all`), supports policy rules through `ip rule`, and most importantly answers the question that `route` cannot: `ip route get DEST` tells you which route the kernel will pick for traffic to `DEST`, accounting for every table and rule simultaneously.

Side-by-side syntax

Task	Legacy (<code>route</code>)	Modern (<code>ip route</code>)
Show routing table (numeric)	<code>route -n</code>	<code>ip route show</code>
Show IPv6 routes	<code>route -A inet6 -n</code>	<code>ip -6 route show</code>
Add a static route	<code>route add -net 10.10.0.0/16 gw 10.0.0.1</code>	<code>ip route add 10.10.0.0/16 via 10.0.0.1</code>
Delete a static route	<code>route del -net 10.10.0.0/16</code>	<code>ip route del 10.10.0.0/16</code>
Set default gateway	<code>route add default gw 10.0.0.1</code>	<code>ip route add default via 10.0.0.1</code>
What route would be used?	Not supported	<code>ip route get 10.20.30.5</code>
Show all tables	Not supported	<code>ip route show table all</code>

3.3 DBA Real-World Scenario

Scenario: replication traffic must use a dedicated 10 Gb interface

Your MySQL primary in a colocation rack has two NICs: a 1 Gb management NIC on bond0 and a 10 Gb replication NIC on bond1. The replica subnet (10.50.0.0/16) must be reached over bond1, but the host's default gateway is on bond0. With route, you could add a static route; with ip route you also gain the diagnostic to prove it took effect:

```
# Add a route forcing replication traffic over bond1
ip route add 10.50.0.0/16 dev bond1 src 10.50.0.7

# Verify which interface the kernel will use for the replica's IP
ip route get 10.50.0.42
# Expected output:
# 10.50.0.42 dev bond1 src 10.50.0.7 uid 0

# After verification, persist the route (NetworkManager example)
nmcli connection modify bond1 +ipv4.routes "10.50.0.0/16"
```

The ip route get step is the part route could never do. Without it, you would have to guess from reading the table and the source-selection rules. With it, you have a single deterministic answer that costs nothing to run before opening replication on the application side.

3.4 When the Legacy Command Still Wins

- **Single-line scripts on embedded or legacy hosts** where route's terser output is easier to grep and the host pre-dates iproute2 (rare in 2026).
- **Cross-platform muscle memory** if your fleet includes BSD; FreeBSD's route command has different semantics but similar surface.

3.5 Comparable Tools

- **mtr** : combines ping and traceroute into a continuously updating per-hop loss and latency view. Invaluable when the routing question is “where does the packet die?” rather than “which route would be picked?”
- **ip rule** : for inspecting and managing policy routing rules. If ip route get returns an unexpected interface, the answer almost always lies in ip rule show.

3.6 Conclusion

Recommendation

Use ip route for inspection, configuration, and especially for ip route get during incident response. Treat route as a museum piece: recognize it on old runbooks, but do not write new ones with it.

Chapter 4 — From netstat to ss

4.1 The Problem

Every DBA has typed `netstat -tulnp` at least a thousand times. It is the canonical answer to “what is listening on port 5432?” and “which process holds that socket?” `netstat` works by parsing `/proc/net/tcp`, `/proc/net/udp`, and friends. That worked when the kernel maintained a few hundred sockets per host. On a modern PostgreSQL or MySQL host with thousands of concurrent connections, `netstat` becomes pathologically slow: it re-reads `/proc` on every invocation, holds the data in user space, and walks every socket linearly. On a busy production host, `netstat -tulnp` can take ten seconds to return.

The deeper problem is that `/proc/net/tcp` is a text interface designed for human reading. The kernel has long offered a richer, faster, binary interface — the `sock_diag` netlink family — but `netstat` does not use it.

4.2 The Modern Solution

`ss` (socket statistics), also part of `iproute2`, uses `sock_diag` exclusively. It is typically an order of magnitude faster than `netstat` on busy hosts, exposes information `netstat` cannot (TCP internals like congestion window, retransmit counts, RTT estimate), and filters at the kernel rather than the client. The filter language is the real productivity multiplier: `ss -t state established '( dport = :5432 or sport = :5432 )'` returns in milliseconds even on hosts with 50,000 sockets.

Side-by-side syntax

Task	Legacy (netstat)	Modern (ss)
All listening TCP/UDP ports	<code>netstat -tulnp</code>	<code>ss -tulnp</code>
All established TCP	<code>netstat -tan grep ESTAB</code>	<code>ss -t state established</code>
Sockets for a specific port	<code>netstat -tan grep :5432</code>	<code>ss -tan '(sport = :5432 or dport = :5432)'</code>
Per-socket TCP internals	Not available	<code>ss -tin</code>
Unix sockets	<code>netstat -x</code>	<code>ss -x</code>
Summary by protocol	<code>netstat -s</code>	<code>ss -s</code>
Routing table	<code>netstat -rn</code>	<code>ip route</code>
Interface statistics	<code>netstat -i</code>	<code>ip -s link</code>

4.3 DBA Real-World Scenario

Scenario: connection storm on a PostgreSQL primary

A monitoring alert fires: PostgreSQL connection count has spiked from a baseline of 200 to 4,800 in three minutes. You need to know whether the storm is from a single application host (a runaway connection pool) or distributed (a real surge). The legacy approach:

```
netstat -tan | grep :5432 | awk '{print $5}' | cut -d: -f1 | sort | uniq -c | sort -rn
```

On a host with 5,000 sockets this command can take 15 seconds — long enough that by the time you have a count, the situation has changed. The `ss` equivalent uses kernel-side filtering and returns immediately:

```
# Count established connections to port 5432, grouped by source IP
ss -tan state established '( sport = :5432 )' \
  | awk 'NR>1 {split($4,a,""); print a[1]}' \
  | sort | uniq -c | sort -rn | head -10

# The TCP internals view (no equivalent in netstat) – find connections
# with high retransmit counts, which usually signal network problems
ss -tin state established '( sport = :5432 )' \
  | grep -E 'retrans:[1-9]'
```

The second command is the gold. It shows TCP-level retransmit counters per connection, which a DBA can use to distinguish “application is opening too many connections” from “network is dropping packets and the application is retrying.” `netstat` physically cannot produce that view.

Performance note

On a host with 50,000 sockets, `ss -tan` typically returns in under 100 ms. `netstat -tan` on the same host can take 8–12 seconds and burns measurable CPU. During an active incident, this is the difference between diagnosis and guessing.

4.4 When the Legacy Command Still Wins

- **netstat -r and netstat -i** remain familiar shorthand for routes and interface counters; `ip route` and `ip -s link` are the modern equivalents.
- **Cross-platform scripts** that must also run on BSD and Solaris should stay with `netstat`; `ss` is Linux-only.
- **Hosts without iproute2** (very old or extremely minimal). In 2026 this is rare.

4.5 Comparable Tools

- **lsof -i** : still excellent when the question is “which processes hold an open file or socket?” and you need richer per-process detail than `ss -p` provides.
- **bpftrace / bpftrace** : when `ss` is not enough — for example, when you need to know who is opening these sockets in real time. The replacement-for-the-replacement, for DBAs who need it.
- **fuser -n tcp 5432** : pinpoint which processes have a specific port open. Underrated complement to `ss`.



4.6 Conclusion

Recommendation

Replace netstat with ss in every DBA runbook this quarter. The performance improvement on busy hosts alone justifies the migration; the TCP internals view (ss -tin) is a permanent operational upgrade that has no netstat equivalent.

Chapter 5 — ARP, Bridges, and Wireless

This chapter consolidates three smaller migrations from net-tools: `arp` → `ip neigh`, `brctl` → `bridge` / `ip link`, and `iwconfig` → `iw`. DBAs touch these less often than `ip` and `ss`, but each has at least one production scenario worth knowing.

5.1 `arp` → `ip neigh`

The Problem

The `arp` command from net-tools shows the kernel's ARP cache — the mapping from IPv4 addresses to MAC addresses for hosts on the same layer-2 segment. Its IPv6 counterpart is the neighbor discovery (ND) cache, which `arp` cannot show at all. Splitting two near-identical concerns across two tools is exactly the kind of mismatch `iproute2` was built to remove.

The Modern Solution

`ip neigh` shows both ARP (for IPv4) and ND (for IPv6) entries in one unified view, with explicit state information (REACHABLE, STALE, FAILED) that the `arp` command underreports.

Task	Legacy	Modern
Show ARP/ND cache	<code>arp -an</code>	<code>ip neigh show</code>
Delete an entry	<code>arp -d 10.0.0.5</code>	<code>ip neigh del 10.0.0.5 dev eth0</code>
Add a static entry	<code>arp -s 10.0.0.5 aa:bb:cc:dd:ee:ff</code>	<code>ip neigh add 10.0.0.5 lladdr aa:bb:cc:dd:ee:ff dev eth0</code>
Show only IPv6 neighbors	Not supported	<code>ip -6 neigh show</code>

DBA Scenario

A virtual IP (VIP) used by your PostgreSQL failover cluster has just moved from the old primary (mac aa:bb:01) to the new primary (mac aa:bb:02). Applications time out connecting to the VIP. The ARP cache on a busy application host still maps the VIP to the old MAC. The fix:

```
# Flush the stale entry; the kernel will re-ARP and pick up the new MAC
ip neigh flush dev eth0 to 10.0.0.100

# Confirm the new mapping
ip neigh show dev eth0 | grep 10.0.0.100
```

5.2 `brctl` → `bridge` / `ip link`

The Problem

brctl is the legacy bridge utility. It pre-dates the modern bridge command (also from iproute2) and the ip link bridge subcommand. On any host running Docker, Kubernetes, libvirt VMs, or LXD containers, you have one or more virtual bridges (docker0, virbr0, cni0). brctl can show them but cannot expose modern bridge features like VLAN filtering, multicast snooping, or per-port flags.

The Modern Solution

ip link is used to create and destroy bridges; bridge is used to inspect and configure their per-port behaviour.

Task	Legacy	Modern
Show bridges	<code>brctl show</code>	<code>bridge link show</code>
Create a bridge	<code>brctl addbr br0</code>	<code>ip link add br0 type bridge</code>
Add interface to bridge	<code>brctl addif br0 eth0</code>	<code>ip link set eth0 master br0</code>
Remove from bridge	<code>brctl delif br0 eth0</code>	<code>ip link set eth0 nomaster</code>
Bridge MAC table	<code>brctl showmacs br0</code>	<code>bridge fdb show br br0</code>

DBA Scenario

A PostgreSQL container on a Docker host cannot reach a sibling container on the same docker0 bridge. brctl show docker0 lists the interfaces; bridge fdb show br docker0 additionally shows whether the sibling's MAC is being learned. If a MAC is missing from the forwarding database, the bridge is silently dropping frames — a diagnosis brctl cannot deliver.

5.3 iwconfig → iw

The Problem

iwconfig is part of the old wireless-tools package, designed for the wireless extensions (WEXT) kernel interface. WEXT was deprecated in favour of nl80211 (a netlink family) more than a decade ago. iwconfig cannot expose WPA3, 802.11ac/ax features, or any modern security configuration.

The Modern Solution

iw is the user-space CLI for nl80211 and is the only correct tool for modern wireless work.

DBA relevance

Wireless is rarely a database concern in production. This pair matters mainly for laptops used as jump boxes or for edge-deployed databases in IoT scenarios. Most DBAs can read this section once and move on.

Task	Legacy (iwconfig)	Modern (iw)
Show wireless interfaces	<code>iwconfig</code>	<code>iw dev</code>



Scan for networks	<code>iwlist scan</code>	<code>iw dev wlan0 scan</code>
Show link quality	<code>iwconfig wlan0</code>	<code>iw dev wlan0 link</code>

5.4 Conclusion for Chapter 5

Recommendation

For ARP issues, `ip neigh` is a strict upgrade — use it. For bridges, learn `bridge` for inspection; `brctl` is dropped from many minimal images. For wireless, `iw` is the only correct choice on modern Linux; `iwconfig` should not appear in a current runbook.

Chapter 6 — DNS Tools: From nslookup to dig

6.1 The Problem

nslookup (name server lookup) was the canonical DNS query tool on most Unix-like systems for decades. It is still installed almost everywhere because muscle memory dies hard. But nslookup is fundamentally a debugging shell that wraps a small subset of DNS queries; it cannot query arbitrary record types cleanly, it cannot show the full DNS response with flags and authority sections, it cannot specify a query class, and its output format has historically varied across implementations (BIND nslookup, Microsoft nslookup, and BusyBox nslookup all produce different text). For scripted use, parsing nslookup output is a fragile mistake.

host is the lighter-weight ISC sibling of nslookup and improves on it, but it still hides most of the DNS response and is sparingly maintained.

6.2 The Modern Solution

dig (domain information groper, from the BIND project) is the DNS query tool that DNS engineers actually use. It returns the full DNS response — header flags, question section, answer section, authority section, additional section, query time, server, and message size — in a format that is both human-readable and consistently scriptable. It supports every record type, custom DNS classes, recursion control, EDNS options, DNSSEC validation chains, and TCP versus UDP transport. dig is the right tool for every DNS question a DBA encounters.

Side-by-side syntax

Task	Legacy (nslookup / host)	Modern (dig)
Forward lookup (A record)	<code>nslookup db-primary.corp</code>	<code>dig db-primary.corp A</code>
Short answer only (scripting)	(nslookup output requires grep)	<code>dig +short db-primary.corp</code>
Reverse lookup	<code>nslookup 10.0.0.5</code>	<code>dig -x 10.0.0.5</code>
Query specific server	<code>nslookup db.corp 8.8.8.8</code>	<code>dig @8.8.8.8 db.corp</code>
MX records	<code>nslookup -type=MX corp.com</code>	<code>dig corp.com MX</code>
All records	<code>nslookup -type=any corp.com</code>	<code>dig corp.com ANY</code>
Trace full delegation path	Not supported	<code>dig +trace db.corp</code>
DNSSEC chain validation	Not supported	<code>dig +sigchase db.corp</code>

6.3 DBA Real-World Scenario

Scenario: applications resolve the database hostname inconsistently

Half your application pods report “unable to resolve db-primary.corp” while the other half connect fine. The DNS team insists nothing has changed. Three minutes of dig answers the question without escalating:

```
# 1. What does the corporate resolver return right now?
dig @10.0.0.53 db-primary.corp +short

# 2. What does an upstream public resolver return?
dig @1.1.1.1 db-primary.corp +short

# 3. Is the response coming from cache or authoritative?
dig db-primary.corp +noall +answer +stats
# Look at the SERVER and QUERY TIME lines, plus the AUTHORITY section

# 4. Trace the delegation to see who is authoritative
dig db-primary.corp +trace
```

Half the time the answer turns out to be a stale record in one resolver's cache, which a +trace immediately surfaces. The other half it is split-horizon DNS returning different answers depending on the source network — also immediately visible because dig prints the SERVER that answered.

6.4 When the Legacy Command Still Wins

- **Minimal containers** (busybox, alpine without bind-tools) often ship nslookup but not dig. The first action in an Alpine debugging container is usually apk add bind-tools to get dig.
- **Interactive shell users** occasionally prefer nslookup's interactive mode, but dig is no less interactive when invoked without arguments.

6.5 Comparable Tools

- **drill** (part of Idns): a dig alternative with slightly cleaner DNSSEC support. Sysadmin favourite on Debian-derived distros.
- **kdig** (part of Knot DNS tools): dig-compatible CLI with first-class DNS-over-TLS and DNS-over-HTTPS support.
- **getent hosts** : the right tool when you want to know what the system resolver library (NSS) returns, including /etc/hosts and any nss-mdns or systemd-resolved layer. Critical for diagnosing differences between dig and what the database client library actually sees.

6.6 Conclusion

Recommendation

Use dig for all DNS troubleshooting. Use getent hosts when you specifically need to verify what the application's resolver library returns. nslookup is acceptable for casual one-shot lookups in environments where dig is not installed.

Chapter 7 — Connectivity Testing: ping, traceroute, tracepath

7.1 The Problem

ping itself has not been replaced and never will be — it is too foundational. But the default invocation of ping on Linux is unbounded; type `ping db.corp` and it runs forever until you Ctrl-C. In a runbook or a Slack incident channel, an unbounded ping is a hazard: copy-pasted into the wrong terminal, it produces a screenful of garbage. The discipline of always passing `-c 4` (or another bounded count) is a small but meaningful modernization.

traceroute is the older path-discovery tool. It usually relies on UDP probes to high ports, which firewalls often drop, producing the familiar wall of asterisks. It also requires elevated privileges on many distributions to use ICMP mode.

7.2 The Modern Solution

For ping: simply commit to bounded invocations. `ping -c 4 -W 2 db.corp` sends exactly four packets with a two-second per-packet timeout. The result is a fixed-shape command suitable for scripts and runbooks.

For traceroute: tracepath (part of `iproute2`) uses UDP probes that do not require elevated privileges, supports IPv6 transparently, and additionally probes path MTU. It is the friendliest first-line trace tool for DBAs.

For deeper investigation, `mtr` (My TraceRoute) combines ping and traceroute into a single, continuously updating display showing per-hop loss percentage and latency. `mtr` is what you reach for when the question changes from “does it work?” to “where does it die?”

Side-by-side syntax

Task	Legacy / unbounded form	Modern / bounded form
Test connectivity	<code>ping db.corp</code>	<code>ping -c 4 -W 2 db.corp</code>
Trace network path	<code>traceroute db.corp</code>	<code>tracepath db.corp</code>
IPv6 path trace	<code>traceroute6 db.corp</code>	<code>tracepath -6 db.corp</code>
Continuous loss/latency	(loop ping in shell)	<code>mtr -rwc 30 db.corp</code>
Path MTU discovery	Not supported by traceroute	<code>tracepath db.corp</code>

7.3 DBA Real-World Scenario

Scenario: intermittent replication lag with no slow query

Your MySQL replica lag oscillates between 0 and 8 seconds with no corresponding spike in transaction volume on the primary. The DBA hypothesis is network loss. Two commands and ten minutes settle it:

```
# Bounded check: is there packet loss right now?
ping -c 100 -i 0.2 -q 10.20.30.5
# The -q (quiet) flag suppresses per-packet output and prints
# only the summary, which includes loss percentage.

# Per-hop visibility: where is the loss?
mtr -rwc 100 -i 0.5 10.20.30.5
# -r report mode, -w wide, -c 100 cycles, -i 0.5 interval
# Save the output. Any hop showing > 1% loss is suspect.
```

Network teams take a per-hop loss report from mtr much more seriously than “the database feels slow.” The bounded ping with 100 packets at 200 ms intervals gives a statistically meaningful loss number in 20 seconds.

7.4 When the Legacy Command Still Wins

- **tracert** **-I (ICMP mode)** still produces better results than tracepath when the destination is behind a firewall that drops UDP but accepts ICMP echo. Requires CAP_NET_RAW or root.
- **Continuous ping for long-running diagnostics** is fine in a dedicated terminal session, with timestamping (ping -O) for log correlation.

7.5 Comparable Tools

- **hping3** : crafted-packet ping, including TCP SYN probes. Useful when ICMP is filtered but the TCP port is reachable (the classic “ping fails but psql connects” paradox).
- **fping** : parallel ping across many hosts at once. The right tool for a cluster-wide reachability check.
- **nping** (from the nmap suite): bounded, scriptable, supports TCP/UDP/ICMP probes.

7.6 Conclusion

Recommendation

Always pass -c to ping in runbooks. Use tracepath for first-line path discovery and mtr when you need per-hop loss statistics over time. traceroute remains a fine tool when firewall rules dictate ICMP mode.

Chapter 8 — TCP Port Connectivity: From telnet to nc

8.1 The Problem

Generations of sysadmins and DBAs use telnet for a purpose telnet was never designed for: testing whether a TCP port is open. `telnet db.corp 5432` returns “Connected to db.corp” if the port accepts connections and a refusal otherwise. It works, but it has three defects. First, telnet is a remote shell protocol from 1969; it is so insecure that most distributions no longer install the client by default. Second, telnet hangs indefinitely on a connection that opens but produces no banner — common for database protocols. Third, the output is human-only; scripting around it is fragile.

8.2 The Modern Solution

`nc` (netcat) — specifically the OpenBSD reimplementation that ships on most modern Linux distributions — handles TCP port testing cleanly with explicit timeout and exit-code semantics. `nc -vz host port` connects, prints “succeeded” or “refused,” exits with status 0 or non-zero, and returns immediately. The `-w` flag adds a connection timeout, so a hung connection no longer hangs your script.

Side-by-side syntax

Task	Legacy (telnet)	Modern (nc)
Test TCP port (single)	<code>telnet db.corp 5432</code>	<code>nc -vz -w 2 db.corp 5432</code>
Test UDP port	Not supported	<code>nc -vzu -w 2 host 53</code>
Scriptable exit code	Not reliable	<code>nc -z host 5432 && echo OPEN</code>
Scan range of ports	Not supported	<code>nc -zv host 5430-5440</code>
Send a probe payload	Limited	<code>echo -e 'GET / HTTP/1.0\r\n' nc host 80</code>

8.3 DBA Real-World Scenario

Scenario: a new firewall rule may have broken replication

Your network team rolled out a new egress firewall policy at 11 PM. At 11:07 the MySQL replica goes lag-positive. The hypothesis is that the replication port (3306) is no longer permitted from the replica subnet to the primary subnet. Three commands settle it:

```
# From the replica host, prove the port reachability hypothesis
nc -vz -w 3 mysql-primary.corp 3306
# Expected on success: 'Connection to mysql-primary.corp 3306 port [tcp/*]
succeeded!'
# If it returns 'timed out' the firewall is dropping the SYN.
# If it returns 'refused' the host is reachable but nothing is listening.
```

```
# Confirm from a sibling host in a different subnet (control test)
nc -vz -w 3 mysql-primary.corp 3306

# Quick scan: are any of the expected replication ports reachable?
for port in 3306 6033 33060; do nc -zv -w 2 mysql-primary.corp $port; done
```

The discrimination between “timed out” and “refused” is the diagnostic gold. telnet produces both as visually similar errors; nc reports them distinctly so you know whether to talk to the network team or the DBA team.

OpenBSD nc vs GNU netcat vs ncat

Three variants of 'nc' coexist. The OpenBSD nc (default on most modern Linux) supports -z and -w as shown. The older GNU netcat uses slightly different flags. ncat (from the nmap suite) is the most feature-rich, supporting TLS, SOCKS, and brokered listeners. Verify your local variant with nc -h before scripting.

8.4 When the Legacy Command Still Wins

- **Interactive protocol exploration** (reading an SMTP, IMAP, or HTTP server's banner conversationally) is still pleasant with telnet. nc works for the same purpose but is less commonly remembered.

8.5 Comparable Tools

- **curl -v telnet://host:port** : surprising but functional; useful when nc is not installed but curl is.
- **tcping / paping** : purpose-built TCP-port pings that loop and report per-attempt latency. Useful for finding flapping ports.
- **psql -h host -p 5432** (or mysql -h, mongo --host): the ultimate proof. If the database client connects, the port works.

8.6 Conclusion

Recommendation

Standardize on nc -vz -w N for all port-reachability tests in DBA runbooks. The combination of exit codes, timeouts, and clear refusal/timeout discrimination eliminates an entire class of false-positive escalations.

Chapter 9 — Secure File Transfer: From ftp to sftp

9.1 The Problem

FTP (File Transfer Protocol) dates from 1971 and sends credentials and data in cleartext. For internal-network backup transfers it survived into the 2000s on the strength of sheer ubiquity, but no production environment in 2026 should still be using it. The Linux ftp client is also gone from most default installs (Debian 12, RHEL 9, Alpine, and others).

9.2 The Modern Solution

sftp (SSH File Transfer Protocol) and scp (secure copy) are bundled with OpenSSH and provide encrypted file transfer over the same channel as ssh. sftp gives an interactive ftp-like shell; scp gives a single-shot cp-like syntax.

Important caveat: as of OpenSSH 9.0 (2022), scp uses the SFTP protocol underneath by default, and the older scp protocol is deprecated. Modern guidance is to prefer sftp for interactive transfer and rsync for bulk synchronization (covered in Chapter 19).

Side-by-side syntax

Task	Legacy (ftp)	Modern (sftp / scp / rsync)
Interactive transfer	ftp host	sftp user@host
Single-file copy	ftp + put	scp file user@host:/path/
Recursive directory copy	ftp + mput (clumsy)	scp -r dir/ user@host:/path/
Resumable transfer	Not supported	rsync --partial --progress src dst
Bulk sync with delete	Not supported	rsync -avz --delete src/ dst/
Bandwidth limit	Variable	rsync --bwlimit=10240 src dst

9.3 DBA Real-World Scenario

Scenario: nightly logical backup ships from primary to backup host

Your nightly pg_dump job produces a 40 GB compressed backup file that must land on a remote backup host. The legacy script uses ftp; it fails sporadically because the network briefly hiccups during the transfer and ftp has no resume capability. The modern replacement:

```
#!/bin/bash
# Nightly PostgreSQL backup ship to off-host storage
BACKUP_FILE=/var/backups/pg/$(date +%F).dump.zst
REMOTE=backup@backup-host.corp:/srv/pg-backups/
```

```
# rsync with resume, compression off (the file is already zstd-compressed),  
# and ssh as the transport. --partial keeps the partial file on failure  
# so the next run resumes from where the last attempt died.  
rsync -av --partial --append-verify --bwlimit=51200 \  
-e 'ssh -i /etc/pg/backup_key -o StrictHostKeyChecking=yes' \  
"$BACKUP_FILE" "$REMOTE"
```

Three modernizations are in play here. The transport is SSH-encrypted (sftp/scp would have been acceptable too). The transfer is resumable (rsync --partial --append-verify). And the bandwidth is capped (--bwlimit=51200, meaning 50 MB/s) so the backup ship does not starve the primary's replication traffic at 03:00.

9.4 When the Legacy Command Still Wins

- **Air-gapped legacy systems** (industrial control, isolated networks) sometimes still mandate FTP for reasons rooted more in compliance inertia than security.
- **Vendor appliances** occasionally expose only FTP. Treat these as defects and request a firmware upgrade or SFTP option.

9.5 Comparable Tools

- **rsync** : the workhorse for any non-trivial transfer; covered fully in Chapter 19.
- **lftp** : a much more capable client that speaks FTP, SFTP, HTTP, and more, with mirror, queue, and segmented download. Useful for one-off bulk transfers where the protocol must be FTP.
- **aws s3 cp / gsutil cp / az storage blob upload** : when the backup destination is cloud storage, the native CLI replaces both ftp and scp.
- **rclone** : rsync-like semantics for sixty-plus cloud and SaaS storage backends. Indispensable for multi-cloud backup strategies.

9.6 Conclusion

Recommendation

Eliminate ftp from every DBA-controlled workflow. Use sftp for interactive transfer, scp for one-shot copies on OpenSSH 9.0+, and rsync (over ssh) for bulk synchronization and backup shipping. For cloud destinations, use the cloud-native CLI or rclone.



Part III — System and Service Management

Every Linux distribution that matters for a database workload in 2026 uses systemd as its init system. The legacy SysV init scripts and the `chkconfig` and `service` commands that managed them have been replaced by `systemctl`, `journalctl`, and `systemd-tmpfiles`. For DBAs this is not optional knowledge. The first thing you do when a database service is misbehaving is look at its unit status and its journal logs, and both of those operations go through systemd-native tools.

Chapter 10 — From SysV init to systemd

10.1 The Problem

SysV init was the Unix init system from 1983 through the early 2010s. Each service was managed by a shell script in `/etc/init.d/`, and three different commands wrapped them awkwardly: `service` (a Red Hat front-end), `/etc/init.d/<svc>` (the direct invocation), and `chkconfig` (the runlevel-based enable/disable manager). The model had structural problems: it had no dependency graph (services started in alphanumeric order of their priority prefix), no parallelism (boot time scaled linearly with service count), no socket activation, no per-service resource limits without writing your own cgroup setup, and crucially, no standard way to capture, rotate, and query service logs.

The Upstart era (Ubuntu's bridge from SysV to systemd) added some of the missing pieces but never reached cross-distribution adoption.

10.2 The Modern Solution

systemd, the init system written by Lennart Poettering and adopted by every major Linux distribution between 2014 and 2017, replaces SysV init with a declarative, dependency-aware, parallel-capable service manager. The user-facing CLI is `systemctl`. For DBAs the relevant feature set is enormous: unit files are declarative (no shell logic to debug), dependencies are explicit and inspectable, every service runs in its own cgroup with memory and CPU accounting included, socket activation enables databases to be started on first connection, and the journal (covered in the next chapter) captures structured logs centrally.

Side-by-side syntax

Task	Legacy	Modern (systemctl)
Start a service	<code>service postgresql start</code>	<code>systemctl start postgresql</code>
Stop a service	<code>service postgresql stop</code>	<code>systemctl stop postgresql</code>
Restart a service	<code>/etc/init.d/postgresql restart</code>	<code>systemctl restart postgresql</code>
Reload config without restart	<code>service postgresql reload</code>	<code>systemctl reload postgresql</code>
Check status	<code>service postgresql status</code>	<code>systemctl status postgresql</code>
Enable on boot	<code>chkconfig postgresql on</code>	<code>systemctl enable postgresql</code>
Disable on boot	<code>chkconfig postgresql off</code>	<code>systemctl disable postgresql</code>
Show all installed services	<code>chkconfig --list</code>	<code>systemctl list-unit-files --type=service</code>
Show what would start	<code>(parse /etc/rc*.d)</code>	<code>systemctl list-dependencies multi-user.target</code>

Task	Legacy	Modern (systemctl)
Show why service failed	(read script + logs)	<code>systemctl status pg && journalctl -u pg</code>

10.3 DBA Real-World Scenario

Scenario: PostgreSQL fails to start after an OOM kill

Overnight, the OOM killer terminated the PostgreSQL process. systemd restarted it (most distributions ship the postgresql.service unit with `Restart=on-failure`), but it failed to come back up. The morning DBA finds this:

```
systemctl status postgresql
# ● postgresql.service - PostgreSQL RDBMS
#   Loaded: loaded (/lib/systemd/system/postgresql.service; enabled)
#   Active: failed (Result: exit-code) since Wed 2026-03-12 02:14:08 IST
#   Process: 1234 ExecStart=/usr/bin/pg_ctlcluster ... (code=exited, status=1)
#
# Mar 12 02:14:08 db01 systemd[1]: postgresql.service: Failed with result 'exit-
code'.

# Get the full failure log from this boot for this unit
journalctl -u postgresql --since '02:00' --until '02:30'

# Inspect the cgroup memory accounting at time of OOM
systemctl show postgresql -p MemoryMax,MemoryCurrent,MemoryHigh

# Apply a temporary override increasing the memory limit
systemctl edit postgresql
# opens an override editor; add:
# [Service]
#   MemoryMax=64G
systemctl daemon-reload && systemctl start postgresql
```

Three things SysV init simply could not do are happening here. `systemctl status` correlates the failure with the journal automatically. `systemctl show` exposes the cgroup memory state the service was running under at failure. `systemctl edit` creates a drop-in override file (`/etc/systemd/system/postgresql.service.d/override.conf`) rather than editing the upstream unit, so package upgrades do not clobber the local change.

DBA-relevant systemctl flags

```
systemctl cat postgresql    — show the merged unit file (origin + drop-ins)
systemctl edit postgresql   — create a drop-in override (preferred over editing the unit)
systemctl daemon-reload    — required after editing unit files manually
systemctl reset-failed postgresql — clear the failed state to allow another start attempt
systemctl --user (per-user units) — rarely used for DB services but worth knowing exists
```

10.4 When the Legacy Command Still Wins

- **service** (the command) is still aliased to `systemctl` on most distros for compatibility. Old runbooks using `'service postgresql start'` will still work — but the moment you need to inspect why something failed, drop the alias.
- **Alpine Linux** uses OpenRC, not `systemd`. `rc-service` and `rc-update` replace `systemctl`. Common in Kubernetes container base images.
- **Container PID 1** : inside a container, `systemd` is rarely PID 1. Tools like `tini`, `dumb-init`, or `s6-overlay` take that role. `systemctl` is not the right tool inside the container.

10.5 Comparable Tools

- **systemd-analyze** : boot performance analysis. `systemd-analyze blame` shows which units took longest to start; `systemd-analyze critical-chain` shows the dependency path that determined boot time. Useful when database services take a surprising amount of time to come up after a host reboot.
- **systemd-cgtop** : top-style view of cgroup resource usage. Effective for finding which `systemd` unit is consuming the most CPU or memory in real time.
- **loginctl**, **machinectl** : the broader `systemd` CLI family. DBAs rarely touch them but should recognize them.

10.6 Conclusion

Recommendation

Adopt `systemctl` as the only service management tool in new runbooks. The combination of structured unit files, automatic log correlation through `journalctl`, and drop-in overrides (`systemctl edit`) is a permanent productivity upgrade. Recognize `service` and `chkconfig` as compatibility shims, not as the right tool.

Chapter 11 — Logging: From /var/log to journalctl

11.1 The Problem

Traditional Unix logging was a story told in plain text files under /var/log/, written by the syslog daemon (syslogd, then rsyslog, then syslog-ng). The model was simple: every service appended messages to a file, logrotate periodically truncated and compressed those files, and operators searched them with grep, less, and awk. That model worked, but it had three weaknesses: messages were unstructured strings (no machine-readable fields), filtering across services required correlation by timestamp, and the boot-to-boot history was discarded by rotation.

11.2 The Modern Solution

The systemd journal, queried with journalctl, stores logs as structured binary records. Each record carries dozens of indexed fields: the unit name, PID, executable, UID, GID, hostname, boot ID, syslog priority, and any custom fields the application emits. The query interface lets you filter on any combination of those fields in one command, across all services on the host, with no grep gymnastics required.

The journal coexists with classic /var/log/syslog on most distributions — systemd-journald can optionally forward to rsyslog — so the migration is additive, not disruptive.

Side-by-side syntax

Task	Legacy	Modern (journalctl)
Tail the system log	<code>tail -f /var/log/syslog</code>	<code>journalctl -f</code>
Show one service's logs	<code>grep postgresql /var/log/syslog</code>	<code>journalctl -u postgresql</code>
Logs since a time	(awk on timestamp)	<code>journalctl --since '1 hour ago'</code>
Logs between times	Awkward	<code>journalctl --since '09:00' --until '10:00'</code>
Logs from previous boot	(read rotated files)	<code>journalctl -b -1</code>
Logs of priority error+	Manual grep	<code>journalctl -p err -b</code>
Logs in JSON for parsing	Not supported	<code>journalctl -u pg -o json</code>
Disk usage of journal	n/a	<code>journalctl --disk-usage</code>
Vacuum old journal data	logrotate	<code>journalctl --vacuum-time=14d</code>

11.3 DBA Real-World Scenario

Scenario: correlate PostgreSQL errors with kernel messages around an incident

At 14:22 a PostgreSQL instance briefly stopped accepting connections. You suspect either an OOM event, a kernel I/O hang, or a connection-storm in the application layer. With classic syslog you would grep three log files and try to line up timestamps. With journalctl:

```
# Show everything from PostgreSQL and the kernel between 14:20 and 14:25
journalctl -u postgresql -k --since '14:20' --until '14:25' --no-pager

# Add priority error and above only, system-wide, for the same window
journalctl -p err --since '14:20' --until '14:25'

# Pipe to jq for structured analysis (every field is indexed)
journalctl -u postgresql --since '14:20' --until '14:25' -o json \
  | jq -r 'select(._SYSTEMD_INVOCATION_ID) | [._REALTIME_TIMESTAMP, .MESSAGE] | @tsv'

# Discover OOM events for the postgres process across all recent boots
journalctl -k -g 'Out of memory' --since '7 days ago'
```

The `-u postgresql -k` combination is the diagnostic that legacy logging cannot deliver in a single command: it interleaves application-level (postgresql) and kernel-level (`-k`) messages on a single timeline, sorted correctly, for one explicit time window. Building the same view from rotated syslog files is an exercise in pain.

Journal persistence and capacity planning

By default systemd-journald keeps the journal in `/run/log/journal` (volatile, lost on reboot) unless `/var/log/journal/` exists. Create that directory to enable persistent storage:

```
mkdir -p /var/log/journal && systemd-tmpfiles --create --prefix /var/log/journal
```

Then set limits in `/etc/systemd/journald.conf`, e.g. `SystemMaxUse=2G`, `MaxRetentionSec=30day`.

For DBA hosts: 2–4 GB is usually enough for 30 days; verbose query logs should go to a dedicated PostgreSQL/MySQL log file, not the journal.

11.4 When the Legacy Command Still Wins

- **Centralized log aggregation** (Splunk, ELK, Loki, Datadog Logs) usually ingests classic syslog/text files or uses a journal-aware shipper (vector, fluent-bit). Your aggregation layer's preferred input format may dictate keeping rsyslog active.
- **Long-running text-only review** (scrolling through last-month evidence for compliance) is sometimes easier in a flat text file than in a journal query loop.

11.5 Comparable Tools

- **rsyslog / syslog-ng**: still the standard for cross-host forwarding, even when the local source is journald (journald forwards to syslog by default on most distros).
- **Inav** (the Log file Navigator): a terminal log viewer with built-in format awareness, SQL-style queries, and timeline view. Excellent for ad-hoc exploration of mixed log files.



- **vector, fluent-bit** : modern log shippers. vector in particular can read from journald, transform, and route to multiple destinations.

11.6 Conclusion

Recommendation

Use journalctl as the primary investigation tool on any systemd-based DBA host. Keep rsyslog active if your aggregation pipeline depends on it. The killer feature is the unified per-service + kernel + priority-filtered timeline (journalctl -u svc -k -p err --since X --until Y).

Chapter 12 — Scheduling: From cron to systemd Timers

12.1 The Problem

cron (and its variants vixie-cron, anacron, fcron) has scheduled Unix jobs since 1975. The model is delightfully simple: a crontab with a five-field schedule and a command. The defects accumulated over fifty years are equally well known: cron has no built-in concept of job state (failure, success, duration), no automatic retry, no dependency between jobs, no resource limits, no log capture beyond MAILTO, and no easy way to express “every 15 minutes but with jitter so a hundred hosts do not fire simultaneously.” Worst of all, when a cron job silently fails, the operator usually finds out by noticing the symptom in the database, not from the scheduler.

12.2 The Modern Solution

systemd timer units pair a .timer file (when to fire) with a .service file (what to run). The service inherits everything systemd offers a long-running service: cgroup accounting, journal logging, restart policy, dependency declarations, environment files, security sandboxing (PrivateTmp, NoNewPrivileges, ProtectSystem). The timer adds Persistent=true (run a missed job after downtime), RandomizedDelaySec= (built-in jitter), and OnUnitInactiveSec= (run N after last completion, never overlapping).

Side-by-side syntax

Task	Legacy (cron)	Modern (systemd timer)
List scheduled jobs	<code>crontab -l</code>	<code>systemctl list-timers</code>
Schedule daily at 02:00	<code>0 2 * * *</code> <code>/usr/local/bin/pg_backup.sh</code>	Timer + service unit (see scenario)
Job ran successfully?	(check MAILTO or logs)	<code>systemctl status pg-backup.service</code>
Job history	(parse syslog)	<code>journalctl -u pg-backup.service</code>
Run job now (out-of-band)	(run command manually)	<code>systemctl start pg-backup.service</code>
Add jitter / spread	Hand-rolled sleep	<code>RandomizedDelaySec=300</code> in .timer
Catch up after downtime	anacron / kludge	<code>Persistent=true</code> in .timer

12.3 DBA Real-World Scenario

Scenario: nightly PostgreSQL backup with reliable logging and retry

The legacy cron line was:



```
0 2 * * * /usr/local/bin/pg_backup.sh >> /var/log/pg_backup.log 2>&1
```

It worked, but failures were noticed only when the on-call DBA happened to read /var/log/pg_backup.log. The systemd timer equivalent — two small files — fixes that.

File 1: /etc/systemd/system/pg-backup.service

```
[Unit]
Description=PostgreSQL nightly logical backup
Wants=network-online.target
After=network-online.target postgresql.service

[Service]
Type=oneshot
User=postgres
Group=postgres
EnvironmentFile=/etc/pg-backup.env
ExecStart=/usr/local/bin/pg_backup.sh
TimeoutStartSec=4h
Nice=10
IOSchedulingClass=best-effort
IOSchedulingPriority=7
PrivateTmp=true
ProtectSystem=strict
ReadWritePaths=/var/backups/pg
```

File 2: /etc/systemd/system/pg-backup.timer

```
[Unit]
Description=Run PostgreSQL nightly backup at 02:00

[Timer]
OnCalendar=*-*-* 02:00:00
RandomizedDelaySec=900
Persistent=true
AccuracySec=1min

[Install]
WantedBy=timers.target
```

Activate and inspect

```
systemctl daemon-reload
systemctl enable --now pg-backup.timer

# Confirm it is scheduled
systemctl list-timers pg-backup.timer

# Run it manually for the first test
systemctl start pg-backup.service

# Inspect the run
systemctl status pg-backup.service
```

```
journalctl -u pg-backup.service --since today
```

Compared to the cron version, this configuration gives explicit dependency on the postgresql service being up (After=postgresql.service), bounded execution time (TimeoutStartSec=4h, so a hung backup is killed rather than running forever), I/O priority lowered so the backup does not starve foreground queries, filesystem sandboxing (ProtectSystem=strict, ReadWritePaths), built-in jitter (RandomizedDelaySec=900, so a hundred hosts do not all start backing up at exactly 02:00:00), automatic catch-up after downtime (Persistent=true), and structured logging in the journal queryable by unit name.

12.4 When the Legacy Command Still Wins

- **Per-user lightweight schedules** where the user is not the system user. `crontab -e` is simpler than systemd user units for one-off shell tasks.
- **Cron's portability** : every Unix-like system has cron; systemd timers are Linux-specific.
- **Container schedulers** : inside a container, Kubernetes CronJob or your platform's native scheduler is almost always the right tool, not systemd timers and not cron.

12.5 Comparable Tools

- **at and batch** : for one-shot scheduling. `at 02:00 < script.sh` is still elegant.
- **Kubernetes CronJob** : when the workload runs in a cluster, the cluster's scheduler is the right surface.
- **Apache Airflow, Dagster, Prefect** : for DAG-style scheduling with dependencies. Overkill for a single backup; appropriate for a multi-step ETL or backup-then-verify-then-ship pipeline.

12.6 Conclusion

Recommendation

Use systemd timers for all production DBA jobs on a systemd host. The combination of journal-based logging, dependency declarations, resource limits, and Persistent=true closes the silent-failure gap that cron leaves wide open. Keep cron for user-scope conveniences and for cross-platform portability.



Part IV — Process and Resource Monitoring

Where Part II covered the network and Part III the service layer, Part IV covers the resource layer: processes, memory, and disk. These are the tools you reach for between the moment a database alert fires and the moment you can name the cause. The modernization here is less about deprecation and more about ergonomics: `top` still works, `ps` still works, `free` still works. The replacements (`htop`, `bttop`, `pgrep`, `free -h`, `duf`, `dust`) give you the same answers faster, with less typing, and with output that fits on a screen during an incident.

Chapter 13 — Process Inspection: ps, pgrep, top, htop, btop

13.1 The Problem

ps and top are not deprecated; they remain installed everywhere and continue to work. The friction is ergonomic. `ps aux | grep postgres` has been the most-typed command in the DBA's repertoire for thirty years, and it has two annoyances: the `grep` matches itself (the famous trailing `| grep -v grep` idiom), and the output truncates the command line at the terminal width, so the actual postgresql arguments often vanish. `top`, similarly, is functional but cramped: scrolling is awkward, killing a process requires looking up the PID, and the per-column meaning is opaque until you have read the man page.

13.2 The Modern Solution

Three replacements split the workload. `pgrep` replaces `ps aux | grep` for the simple case of “give me PIDs matching a name.” `htop` replaces `top` with mouse support, in-place scrolling, tree view, signal-by-keystroke, per-core CPU bars, and customizable columns. `btop` adds richer visualization (CPU per-thread bars, network throughput, disk I/O over time), all in the terminal.

Side-by-side syntax

Task	Legacy	Modern
Find PIDs of a process by name	<code>ps aux grep postgres grep -v grep</code>	<code>pgrep -a postgres</code>
Show full command line	<code>ps -ef grep postgres</code>	<code>pgrep -af postgres</code>
Count matching processes	<code>ps aux grep -c postgres</code>	<code>pgrep -c postgres</code>
PIDs of one user's processes	<code>ps -u postgres</code>	<code>pgrep -u postgres</code>
Send signal by name	<code>kill \$(ps aux grep ...)</code>	<code>pkill -HUP postgres</code>
Interactive monitor	<code>top</code>	<code>htop</code>
Modern monitor with I/O graphs	<code>top</code>	<code>btop</code>
Process tree view	<code>pstree</code>	<code>htop (F5)</code>

13.3 DBA Real-World Scenario

Scenario: identify and gracefully restart a stuck PostgreSQL backend

A monitoring alert reports one PostgreSQL backend stuck for 45 minutes on a query. You want to identify the backend by `application_name`, capture its full command line, and send it a `SIGTERM` only if it matches a specific pattern.


```
# Step 1: list all postgres backends with full command line
pgrep -af 'postgres:.*idle in transaction'
# Example output:
# 18342 postgres: app_user analytics 10.0.5.7(45628) idle in transaction

# Step 2: count how many backends are stuck this way
pgrep -c -f 'postgres:.*idle in transaction'

# Step 3: send SIGTERM to those matching, with confirmation (dry-run first)
pgrep -af 'postgres:.*idle in transaction.*app_user'

# Only after confirming the list is correct:
pkill -TERM -f 'postgres:.*idle in transaction.*app_user'

# Step 4: monitor the recovery
htop -u postgres
```

Production safety with pkill

pkill matches process names by pattern. A pattern that is too broad will signal more processes than you intended.

Always pgrep -af with the same pattern first and read the list before pkill.

On a PostgreSQL host, prefer `SELECT pg_terminate_backend(pid)` over `kill -TERM` whenever possible. The database has more context about what it can safely terminate.

13.4 When the Legacy Command Still Wins

- **Scripts that must be POSIX-portable** should still use `ps`; `pgrep` is a `procps` utility, not POSIX.
- **top is universally installed** and works over the most constrained serial console. `htop` and `btop` need richer terminal capabilities.
- **ps with custom -o formats** (e.g. `ps -eo pid,pmem,rss,vsz,etime,comm`) is unmatched for one-shot tabular output destined for further `awk` processing.

13.5 Comparable Tools

- **atop** : records full system state at intervals and lets you scroll backward in time. Critical for post-mortem analysis when you arrive after the incident is over.
- **nmon** : a long-time AIX export to Linux; comprehensive resource overview.
- **glances** : a Python-based monitor that exposes a web UI and integrates with InfluxDB / Prometheus. Useful when the host has no graphical dashboard.
- **bottom (btm)** : Rust-based, similar to `btop`, with a different visual style. Worth comparing.

13.6 Conclusion

Recommendation



Replace `ps aux | grep` with `pgrep -af` in DBA muscle memory; the difference compounds with every keystroke saved during an incident. Install `htop` and `btm` on every host you administer and pick the one that feels right. Keep `ps` for scripts and post-mortem one-shots.

Chapter 14 — Memory and Disk: free, duf, dust

14.1 The Problem

Three of the most-typed DBA commands — `free`, `df`, and `du` — produce correct but operationally noisy output by default. `free` shows memory in kibibytes (`free -m` fixed it for a generation; `free -h` does so more compactly). `df` shows disk usage in 1K blocks unless told otherwise, and lists every filesystem including `tmpfs` and `squashfs` noise that does not interest a DBA. `du -sh *` is informative but cripplingly slow on large data directories and has no per-directory visualization.

14.2 The Modern Solution

For memory, `free -h` is the only modernization needed; `-h` (human-readable, automatic unit selection) and `-w` (split buffers and cache columns) are the flags worth committing to memory.

For disk, two specialized tools have emerged. `duf` (Disk Usage / Free) renders the equivalent of `df -h` in a coloured, grouped table that filters out the noise filesystems by default. `dust` (`du -- replacement`) shows directory size hierarchically with inline bar graphs and is genuinely faster than `du` on large trees because it parallelizes the traversal.

Side-by-side syntax — memory

Task	Legacy	Modern
Memory in MB	<code>free -m</code>	<code>free -h</code>
Split cache from buffers	<code>free -m</code>	<code>free -hw</code>
Continuously updating	<code>watch -n 1 free -m</code>	<code>free -hs 1</code>

Side-by-side syntax — disk

Task	Legacy	Modern
Filesystem usage	<code>df -h</code>	<code>duf</code>
Local filesystems only	<code>df -hl</code>	<code>duf --only local</code>
Single filesystem detail	<code>df -h /var/lib/postgresql</code>	<code>duf /var/lib/postgresql</code>
Top space consumers in dir	<code>du -sh * sort -hr head</code>	<code>dust</code>
Depth-limited tree	<code>du -h --max-depth=2</code>	<code>dust -d 2</code>
Just the total of a dir	<code>du -sh /var/lib/pg</code>	<code>dust -s /var/lib/pg</code>

14.3 DBA Real-World Scenario

Scenario: a PostgreSQL host is approaching disk full

Monitoring reports `/var/lib/postgresql` is at 89%. You have minutes before the writer starts failing. The classic `du` sequence would take 30 seconds on a multi-terabyte data directory; `dust` returns in a fraction of that time and shows you the breakdown visually:

```
# Where is the space going? Top-level summary
duf --only local

# Within the PostgreSQL data directory, what is the biggest consumer?
dust -d 2 /var/lib/postgresql/15/main
# Output (truncated):
# 4.2T ┌── base/16384          <-- the biggest database
# 890G ┌── pg_wal              <-- WAL accumulation
# 120G └── base/16385
# ...

# Confirm WAL retention is the cause (a common archival failure pattern)
ls -lhS /var/lib/postgresql/15/main/pg_wal | head -20

# If WAL is the cause, check archiver / replication slot state in psql:
psql -c 'SELECT slot_name, active, pg_size_pretty(
    pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS retained
FROM pg_replication_slots ORDER BY 3 DESC;'
```

In an active incident, `dust`'s parallelized scan returning in 5 seconds versus `du`'s 30 seconds is not cosmetic — it is half the time-to-diagnosis. The bar-graph visualization also makes it instantly obvious whether you have one elephant directory or a thousand small ones, which determines whether the recovery is “delete one file” or “archive a whole branch.”

14.4 When the Legacy Command Still Wins

- **Minimal containers and recovery environments** ship `df` and `du`, not `duf` and `dust`. The first command you type in a recovery shell is still `df -h`.
- **Scriptable output** is more stable from `df --output=source,size,used,avail,target`. `duf` has a JSON output (`--json`) for the same purpose, but `df`'s text format is more universally parseable.
- **`du -B 1`** (exact bytes) and `du --apparent-size` are still the right tools for precise reporting where modern bar-graph aesthetics would actually obscure the answer.

14.5 Comparable Tools

- **`ncdu`** (NCurses Disk Usage): interactive disk-usage browser. The DBA-friendly midpoint between `du` and `dust`, with the ability to navigate, mark, and delete from inside the UI. Installed by default on many distributions.
- **`smem`**: memory accounting that correctly attributes shared memory (so the postgres backend numbers in `top` do not look like every connection is using gigabytes).
- **`iostat`, `iotop`**: when the question is not how full but how fast — covered briefly in the migration strategy chapter.

14.6 Conclusion



Recommendation

Always pass `-h` to `free`. Install `duf` and `dust` on hosts you administer for daily use; keep `df` and `du` in muscle memory for minimal recovery environments. Install `ncdu` on any host where you regularly hunt for space; the interactive navigation is unmatched.



Part V — File Operations and Text Processing

The commands in this part are not deprecated and never will be. `ls`, `cat`, `less`, `find`, `grep`, `sed`, and `awk` are foundational Unix vocabulary; a DBA who does not know them is missing a part of the job. The replacements covered here — `eza`, `bat`, `fd`, `ripgrep`, `sd`, `miller` — are quality-of-life upgrades and, in the case of `ripgrep` and `fd`, dramatic performance upgrades. The recommendation throughout this part is the same: install the modern tool, alias it where you find that helpful, and keep the legacy command in muscle memory for portability.

Chapter 15 — File Listing: From ls to eza

15.1 The Problem

ls is forty years old and never wrong, but its defaults are showing their age. Without flags it is plain monochrome text; with flags it requires you to remember roughly twenty-three letter combinations. Colour output (`--color`) is on by default in most distributions, but the colours encode only a small set of categories. There is no Git status integration, no file-type icons, no tree view without the separate `tree` command, and the time formatting (modification time as either short or long ISO depending on age) trips first-time scripters reliably.

15.2 The Modern Solution

eza is a Rust-written modern replacement for ls. It is the actively maintained fork of exa, which was archived by its original author in 2023 (eza is now the upstream every distribution packages). eza adds: file-type colours that distinguish between extensions, file-type icons (with `--icons`, requires a Nerd Font in the terminal), a built-in tree view (`--tree`), and per-file Git status integration (`-l --git`) that shows whether each file is modified, staged, or ignored. It reads more like a modern file manager than a 1979 `ls` command.

Maintenance status note

exa, the original Rust-based ls replacement, was deprecated by its sole maintainer in mid-2023. eza is the community-maintained fork and is the right tool to install on a new system. Where the legacy table in this book lists 'exa', read 'eza' — the command names are interchangeable on most distributions and the project continues forward under the eza name.

Side-by-side syntax

Task	Legacy (ls)	Modern (eza)
Basic listing	<code>ls</code>	<code>eza</code>
Long listing	<code>ls -la</code>	<code>eza -la</code>
With icons (Nerd Font)	Not supported	<code>eza -la --icons</code>
Tree view	<code>tree -L 2</code>	<code>eza --tree --level=2</code>
With Git status	Not supported	<code>eza -l --git</code>
Group dirs first	<code>ls --group-directories-first</code>	<code>eza --group-directories-first</code>
Sort by size	<code>ls -laS</code>	<code>eza -la --sort=size</code>
Show only files	<code>ls -p grep -v /</code>	<code>eza -f files</code>

15.3 DBA Real-World Scenario

Scenario: navigating a PostgreSQL data directory during an investigation

A DBA exploring an unfamiliar PostgreSQL data directory wants to see structure at a glance: which databases are largest, which files have been modified recently, and what the tablespace layout looks like.

```
# Tree view, two levels deep, with file-size summary
eza --tree --level=2 --long --total-size /var/lib/postgresql/15/main

# Sort the base/ directory by modification time to find recent activity
eza -l --sort=modified --reverse /var/lib/postgresql/15/main/base/16384 | head

# The 'long' format with --header gives a column header row
eza -lh --header /var/lib/postgresql/15/main/pg_wal | head -30
```

None of these answer something `ls` cannot deliver. They are all faster to type and easier to read. For a single command run once, the difference is trivial; for a DBA who runs these patterns thirty times a day, it accumulates.

15.4 When the Legacy Command Still Wins

- **Universal availability** : `ls` is on every Unix system; `eza` is on none by default and may not be installable in restricted environments.
- **Scripts must use `ls` or `stat`** : parsing `ls` output is famously fragile, but at least its format is widely understood. `eza`'s `--json` output is the right tool for serious parsing, but `stat --printf` is usually better than both.
- **Coreutils familiarity** : `ls` flags transfer to `find`, `du`, and other coreutils tools. `eza` has its own flag dialect.

15.5 Comparable Tools

- **lsd (LSDeluxe)**: Rust-based, similar feature set, slightly different visual style.
- **broot** : interactive tree navigator with fuzzy filtering; useful for exploring deep directory trees.
- **fzf with `$(find . -type f)`** : when the goal is to pick a file, `fzf` often replaces `ls` entirely.

15.6 Conclusion

Recommendation

Install `eza` on your workstation and any host you frequently log into. Aliasing `ls=eza` is reasonable but optional; many DBAs keep both available and reach for `eza` explicitly for tree views and Git-aware listings. Do not use `eza` in portable scripts; use `stat` or `find` for that.

Chapter 16 — File Viewing: From cat / less to bat

16.1 The Problem

cat is not broken. It is the smallest, most reliable file-printing tool in Unix. But for the common DBA task of reading a configuration file (postgresql.conf, my.cnf, pg_hba.conf, a Python migration script, a SQL file), it prints plain text without syntax highlighting, line numbers, or paging. less adds paging and search but no highlighting. The result is that DBAs frequently scroll past the line that matters because there is no visual cue separating a comment from a key directive.

16.2 The Modern Solution

bat (sometimes invoked as batcat on Debian / Ubuntu to avoid a name collision) is a Rust-written cat replacement with syntax highlighting for over two hundred languages, automatic paging through less when output is longer than the terminal, line numbers, and Git diff markers in the gutter. When piped, bat detects the non-terminal output and degrades to plain cat behaviour automatically, so existing pipelines do not break.

Side-by-side syntax

Task	Legacy	Modern (bat)
Print file	<code>cat postgresql.conf</code>	<code>bat postgresql.conf</code>
No paging, plain cat-like	<code>cat file.txt</code>	<code>bat --paging=never --plain file.txt</code>
Show only specific lines	<code>sed -n '120,140p' file</code>	<code>bat --line-range 120:140 file</code>
Concatenate without highlight	<code>cat a.sql b.sql > c.sql</code>	<code>bat --plain --paging=never a.sql b.sql > c.sql</code>
Highlight a non-detected file	n/a	<code>bat -l sql my_script.txt</code>
Page through long file	<code>less long.log</code>	<code>bat long.log</code>

16.3 DBA Real-World Scenario

Scenario: reviewing pg_hba.conf changes from a configuration drift report

Your configuration management tool has flagged that pg_hba.conf was modified on a production host outside the deployment pipeline. You need to compare the on-disk version against the version in version control, highlighting differences and showing the surrounding context.

```
# View with syntax highlighting (conf format is auto-detected)
bat /etc/postgresql/15/main/pg_hba.conf
```

```
# View only the host-based rules section (lines 80-120)
bat --line-range 80:120 /etc/postgresql/15/main/pg_hba.conf

# Show two files side by side via diff piped to bat for highlighting
diff -u /etc/postgresql/15/main/pg_hba.conf ./pg_hba.conf.git \
| bat -l diff --paging=never
```

The Git diff style highlighting in the third command — additions in green, deletions in red, contextual unchanged lines in white — makes a diff legible in a terminal in a way that plain diff output is not. This matters less for one diff and a great deal for thirty diffs across a cluster of hosts.

Naming gotcha

On Debian and Ubuntu the binary is named batcat (the bat name was already taken by an unrelated package). Create an alias if you want:

```
alias bat='batcat'
```

On RHEL, Fedora, Arch, and macOS Homebrew, the binary is simply bat.

16.4 When the Legacy Command Still Wins

- **Pipelines that depend on cat being a no-op** (useless use of cat aside, the canonical “cat | something” pattern is everywhere). bat respects pipe semantics but adds startup overhead.
- **Tiny rescue environments** : bat is not in busybox; cat is.
- **Mass file printing** : for printing 1,000 files in a loop, cat is materially faster.

16.5 Comparable Tools

- **less** : still the default \$PAGER on most systems. less +F /var/log/file behaves like tail -f with the ability to scroll backwards. Often paired with bat (bat sets less as its pager by default).
- **moar** : a Go-based less replacement with friendlier defaults.
- **delta** (git-delta): the syntax-highlighting diff viewer used to render git diff. Set core.pager = delta in ~/.gitconfig for an immediate upgrade to the git diff experience.

16.6 Conclusion

Recommendation

Install bat on workstations and frequently used hosts. Use cat in scripts and pipelines. Combine bat with delta for a syntax-highlighted git diff experience that makes configuration drift reviews significantly less painful.

Chapter 17 — Finding Files: From find to fd

17.1 The Problem

`find` is one of the most powerful tools in Unix and also one of the least pleasant to type. The syntax requires explicit `-type`, `-name` with quoted patterns, careful ordering of predicates, and idiomatic `-exec ... \;` or `-exec ... +;`; the difference between `-name` and `-path` is non-obvious; and the default behaviour traverses every filesystem and every hidden directory unless told otherwise. The first time a DBA runs `find / -name 'postgresql.conf'` on a host with network filesystems mounted, they learn about `-xdev` the hard way.

17.2 The Modern Solution

`fd` is a Rust-written `find` replacement with sensible defaults: it ignores `.git` and other common noise directories by default, respects `.gitignore`, parallelizes the traversal, colours output, treats the first positional argument as a regex against the filename, and does not require explicit `-name` or quoting in the common case. The result is that `fd config` matches every file containing 'config' in its name across the tree, in one short command.

Side-by-side syntax

Task	Legacy (find)	Modern (fd)
Find by name	<code>find . -name '*.conf'</code>	<code>fd '.conf\$'</code>
Find by extension	<code>find . -name '*.sql'</code>	<code>fd -e sql</code>
Case-insensitive	<code>find . -iname '*.SQL'</code>	<code>fd -i sql</code>
Find directories only	<code>find . -type d -name backups</code>	<code>fd -t d backups</code>
Exclude a directory	<code>find . -path './node_modules' -prune -o -print</code>	<code>fd -E node_modules</code>
Find and exec	<code>find . -name '*.log' -exec gzip {} \;</code>	<code>fd -e log -x gzip</code>
Find modified in last 7 days	<code>find . -mtime -7</code>	<code>fd --changed-within 7d</code>
Limit depth	<code>find . -maxdepth 3</code>	<code>fd -d 3</code>

17.3 DBA Real-World Scenario

Scenario: locate all PostgreSQL configuration files across a fleet of hosts

During an audit you must enumerate every `postgresql.conf`, `pg_hba.conf`, and `recovery.signal` file under `/etc` and `/var/lib` on each host.

```
# Find all postgresql.conf files, ignoring symlinks and respecting filesystem
boundaries
fd -t f --one-file-system postgresql.conf /etc /var/lib

# Find every PostgreSQL configuration file by extension
fd -e conf --search-path /etc/postgresql --search-path /var/lib/postgresql

# Find and act: print sha256sum of each pg_hba.conf
fd pg_hba.conf -t f -x sha256sum

# The find equivalent (verbose by comparison)
find /etc /var/lib -xdev -type f \( -name 'postgresql.conf' -o -name 'pg_hba.conf'
\) \
    -exec sha256sum {} \;
```

The `fd` command takes one line. The `find` equivalent takes three and contains two punctuation traps (the escaped parentheses for the OR clause and the escaped semicolon at the end of `-exec`). For a DBA scripting an audit across a hundred hosts, the cognitive savings of `fd` are worth more than the raw performance difference.

17.4 When the Legacy Command Still Wins

- **Complex predicates** (combining `-newer`, `-size`, `-user`, `-group` with arbitrary logic) are still cleaner in `find`. `fd` is optimized for the common case, not the exhaustive one.
- **POSIX shell scripts** should use `find`. `fd` is not POSIX and not installed by default anywhere.
- **Hidden file / dotfile traversal** : `find` shows everything by default; `fd` hides dotfiles unless you pass `-H`. The default difference catches people the first time.

17.5 Comparable Tools

- **locate / mlocate** : indexed file search, instantaneous response, but only as fresh as the last `updatedb` run. Useful for repeated lookups, not for new files.
- **ripgrep with --files** (`rg --files`): produces the list of files `rg` would search, which is often what you actually want before grepping. `fd` and `rg --files` overlap in practice.

17.6 Conclusion

Recommendation

Install `fd` alongside `find`. Use `fd` for interactive exploration and one-shot scripts; use `find` for portable scripts and complex predicate logic. The default `.gitignore`-aware behaviour of `fd` is the single biggest reason to adopt it.

Chapter 18 — Content Search: From grep to ripgrep

18.1 The Problem

grep -r is the universal text search tool. It works, it is everywhere, and its defaults are bad. It searches binary files by default (often producing a screen of garbled output before you can Ctrl-C), it traverses .git directories, it does not respect .gitignore, and on large trees it is single-threaded and slow. The DBA who searches a 200 GB log archive for a specific error string with grep -r learns the difference between “usable” and “fast.”

18.2 The Modern Solution

ripgrep (binary name rg) is a Rust-written grep replacement that combines a fast regex engine, parallel directory traversal, automatic binary file detection (skipped by default), and .gitignore awareness. Benchmarks routinely show ripgrep 5–10x faster than grep -r on large trees and an order of magnitude faster than ack or the silver searcher. The defaults are also smarter: rg pattern in a directory does what grep -rl --exclude-dir=.git pattern does, without the flags.

Side-by-side syntax

Task	Legacy (grep)	Modern (rg)
Search recursively	grep -rn 'pattern' .	rg 'pattern'
Case-insensitive	grep -rni 'pattern' .	rg -i 'pattern'
Whole word	grep -rnw 'pattern' .	rg -w 'pattern'
Search only one file type	find . -name '*.sql' xargs grep	rg -t sql 'pattern'
Exclude file type	(complex find chain)	rg -T log 'pattern'
With surrounding context	grep -B 3 -A 3 'pattern' file	rg -C 3 'pattern' file
Show only filenames	grep -l 'pattern' -r .	rg -l 'pattern'
Multiline regex	(awkward)	rg -U '(?s)pattern.*?end'

18.3 DBA Real-World Scenario

Scenario: hunt for a specific deadlock signature across last month's PostgreSQL logs

Your backup process has uploaded thirty days of compressed PostgreSQL logs to a local analysis directory. The error you are hunting is a deadlock_detected line with a specific relation name. grep would happily scan the directory, but it would also try to decompress nothing and skip the compressed files silently.

```
# ripgrep with type filter for log files, context, and compressed file support
rg --type log -C 5 'deadlock detected.*orders_2026' /srv/pg-logs/

# Search inside .gz logs too (ripgrep auto-detects when --search-zip is set)
rg --search-zip --type-add 'pglog:*.{log,log.gz,log.zst}' -t pglog \
  -C 5 'deadlock detected.*orders_2026' /srv/pg-logs/

# Count occurrences per file, sorted descending
rg --search-zip -c 'deadlock detected' /srv/pg-logs/ | sort -t: -k2 -rn | head

# Pipe to bat for highlighted context
rg --json --search-zip -C 3 'deadlock detected' /srv/pg-logs/ | head
```

Speed difference, in numbers

Searching a 50 GB directory of mixed plain-text and gzip-compressed log files:

grep -rni 'deadlock' . on the same hardware: about 18 minutes (and silently skips .gz)

rg --search-zip -i 'deadlock': about 90 seconds (and reads .gz)

The reason is twofold: ripgrep parallelizes the file traversal, and its regex engine (built on the Rust regex crate) is fundamentally faster than GNU grep's POSIX engine for most patterns.

18.4 When the Legacy Command Still Wins

- **Portable scripts** : grep is POSIX and is everywhere. rg is not.
- **PCRE-only patterns** : rg supports a powerful regex dialect but does not enable PCRE features by default. Use rg -P or stick with grep -P.
- **Stream filters in pipelines** : grep is the canonical line filter in pipe-heavy scripts. ripgrep works as a stream filter (rg < file) but its strengths shine on directory traversal.

18.5 Comparable Tools

- **ack** (the Perl-based code search tool that prefigured rg). Still installed in some environments.
- **the_silver_searcher (ag)** : fast C-based search; ripgrep has eclipsed it in most benchmarks but ag is widely deployed.
- **ugrep** : grep-compatible CLI with extra features (fuzzy match, archive support). A good replacement when scripts depend on grep flag compatibility.

18.6 Conclusion

Recommendation

Adopt ripgrep for all interactive content search. The combination of speed, smart defaults, and --search-zip for compressed logs makes it materially better for DBA log-hunting work. Keep grep for portable scripts and one-line pipe filters.

Chapter 19 — Text Manipulation: sed → sd, awk → mlr

19.1 The Problem

sed is a stream editor with a domain-specific language nearly as old as Unix itself. For substitution it is unmatched, but the substitution syntax (sed 's/pattern/replacement/g') is dense, escape-heavy, and trips users on every metacharacter that needs to be escaped or unescaped. awk is a complete text-processing programming language with thirty years of accumulated wisdom and a slightly weird default behaviour for tabular data: it splits on whitespace, not on commas or tabs, and requires explicit -F to behave like a CSV reader. For DBAs who routinely process CSV exports, awk's whitespace bias is a constant source of small bugs.

19.2 The Modern Solution

sd is a Rust-written sed replacement focused on the most common sed use case: regex find-and-replace. It uses the same regex flavour as ripgrep, treats the pattern as a regex by default (no -E required), and uses {} interpolation for backreferences. The result is that sd 'foo' 'bar' file is exactly what you typed, without sed's leading s, trailing /g, or escape gymnastics.

mlr (Miller) is the awk-for-tabular-data tool. It reads and writes CSV, TSV, JSON, JSON lines, and a half-dozen other formats natively; its verbs (cut, filter, stats1, join, sort) compose into pipelines that read like SQL. For any DBA who routinely transforms CSV between systems, Miller is more useful than awk after the first week of practice.

Side-by-side syntax — sed → sd

Task	Legacy (sed)	Modern (sd)
Substitute in stream	sed 's/old/new/g'	sd 'old' 'new'
Substitute in place	sed -i 's/old/new/g' file	sd 'old' 'new' file
Match a regex	sed -E 's/v[0-9]+/vX/' file	sd 'v\d+' 'vX' file
Backreferences	sed -E 's/(foo)bar/\1baz/'	sd '(foo)bar' '\${1}baz'
Multiline / DOTALL	(awkward)	sd -p '(?s)A.*?B' 'X'

Side-by-side syntax — awk → mlr

Task	Legacy (awk)	Modern (mlr)
Select columns from CSV	awk -F, '{print \$1,\$3}'	mlr --csv cut -f col1,col3 in.csv
Filter rows	awk -F, '\$3>100' file	mlr --csv filter '\$col3>100' in.csv

Task	Legacy (awk)	Modern (mlr)
Sum a column	<code>awk -F, '{s+=\$3} END{print s}'</code>	<code>mlr --csv stats1 -a sum -f col3 in.csv</code>
Group-by aggregation	(loop)	<code>mlr --csv stats1 -a sum -f amount -g region in.csv</code>
CSV → JSON	(manual)	<code>mlr --icsv --ojson cat in.csv</code>
Join two CSV files	(complex)	<code>mlr --csv join -j id -f a.csv b.csv</code>

19.3 DBA Real-World Scenario

Scenario: clean and merge per-table size CSVs from a fleet of hosts

Each PostgreSQL host emits a daily CSV of table sizes (`table_sizes.csv`) with columns `host,db,table,bytes`. You need to (a) drop rows where bytes < 1 MB, (b) sum bytes per database across hosts, and (c) write the result as JSON for ingestion by your dashboard.

```
# Concatenate all CSVs, filter, group, output as JSON – one Miller invocation
mlr --c2j \
  filter '$bytes >= 1048576' \
  then stats1 -a sum -f bytes -g db \
  then label db,total_bytes \
  cat /srv/reports/2026-*/table_sizes.csv > /srv/reports/db_totals.json

# Equivalent awk version is uncomfortably long and fragile around quoting
```

Scenario: rename a configuration directive in postgresql.conf in place

```
# sed equivalent (with escape: # is the comment marker, no slash conflict here)
sed -i 's/^#*shared_buffers = .*/shared_buffers = 8GB/'
/etc/postgresql/15/main/postgresql.conf

# sd equivalent – no comment about escaping, no -E
sd '^#*shared_buffers = .*' 'shared_buffers = 8GB'
/etc/postgresql/15/main/postgresql.conf
```

19.4 When the Legacy Command Still Wins

- **sed is POSIX** : every Unix system has it. `sd` is not.
- **Complex sed scripts** with multiple address ranges, hold space, and branch labels have no clean `sd` equivalent. `sd` is for find-and-replace, not for arbitrary stream editing.
- **awk for one-liners** on whitespace-separated logs (web access logs, database slow logs in their native format) remains the right tool. `mlr`'s strength is tabular formats with explicit headers.

19.5 Comparable Tools

- **csvkit** (csvcut, csvsql, csvjson, csvgrep): Python-based CSV toolkit. csvsql is particularly DBA-friendly: it lets you run SQL queries against CSV files using SQLite or PostgreSQL as the backend.
- **xsv** : Rust-based CSV toolkit, very fast for large files.
- **jq** : indispensable for JSON manipulation; pairs naturally with mlr.
- **q** : run SQL queries against text files (CSV, TSV) directly from the shell. Useful for ad-hoc analysis when you do not want to load data into a real database.

19.6 Conclusion

Recommendation

Keep sed and awk in muscle memory — they are not going away. Add sd for everyday find-and-replace work where its cleaner syntax matters. Add mlr (Miller) for any CSV / TSV / JSON transformation; it pays for itself the first time you write a five-verb pipeline that would have been a forty-line awk script.

Part VI · HTTP, Transfer, and Compression

The four chapters in this part cover the commands a DBA reaches for when data leaves a host or arrives on one: hitting a database REST endpoint, downloading a binary, copying a dump file across a slow link, and compressing a backup before it hits object storage. Each pair below is selected because the modern replacement changes the economics — usually compression ratio, throughput, or restart-after-failure behavior — in a way that directly affects how long a DBA waits, and therefore how long the database waits.

Chapter 20 · curl → HTTPie

The Problem

Modern databases expose more than a wire protocol. PostgreSQL ships with PostgREST in many deployments; MongoDB Atlas, Cassandra DataStax, Snowflake, BigQuery, and every cloud-managed database speak HTTPS for administration, metrics, and increasingly for the data plane itself. The DBA who never has to write JSON over HTTP no longer exists. curl, written by Daniel Stenberg in 1996, is the universal HTTP client — but its output is raw, its JSON ergonomics are nonexistent, and a complex POST with headers and a body requires a string of flags that does not survive a code review.

The Modern Solution

HTTPie is a Python-based HTTP client designed for humans. It defaults to JSON for both request and response, pretty-prints and colorizes output, supports session files, and uses a key=value DSL for headers and body that reads like a description rather than a shell incantation. For a DBA who needs to fetch metrics from a Prometheus endpoint, post a payload to a database admin API, or test an OAuth-protected reporting service, HTTPie compresses what was a multi-line curl into a single readable command.

Side-by-Side Comparison

Task	curl	HTTPie
GET JSON endpoint	<code>curl -s https://api/x jq .</code>	<code>http https://api/x</code>
POST JSON body	<code>curl -X POST -H 'Content-Type: application/json' -d '{"k":"v"}' https://api/x</code>	<code>http POST https://api/x k=v</code>
Custom header	<code>curl -H 'X-Token: abc' https://api/x</code>	<code>http https://api/x X-Token:abc</code>
Basic auth	<code>curl -u user:pass https://api/x</code>	<code>http -a user:pass https://api/x</code>
Download file	<code>curl -o file.dump https://host/file.dump</code>	<code>http --download https://host/file.dump</code>
Follow redirects	<code>curl -L https://host/x</code>	<code>http --follow https://host/x</code>
Show only headers	<code>curl -I https://host/x</code>	<code>http --headers https://host/x</code>

Real-World DBA Scenario

A PostgreSQL platform team exposes a PostgREST endpoint for an internal reporting application. After a deployment, the application team reports that POST requests to `/rpc/refresh_materialized_view` are returning 400. The DBA needs to reproduce the failing call from a bastion host to confirm whether the problem is in the endpoint or in the application's serialization layer. With curl the exact reproduction takes three attempts to get the quoting right; with HTTPie the same request is two lines:

```
$ http POST https://api.internal/rpc/refresh_materialized_view \  
  Authorization:'Bearer eyJhbGc...' \  
  view_name=order_summary_daily
```

HTTPIe color-codes the response status, prints the response headers in green, and pretty-prints the JSON body — the DBA sees in one screen that the endpoint is returning `{"hint":"column view_name does not exist"}` and that the application is sending the wrong parameter name. The same diagnosis with curl typically requires piping through jq and reading uncolored output.

Production tip — in a Kubernetes init container or a minimal Alpine image, HTTPIe's Python dependency is a footprint hit. For container probes and CI scripts, stay with curl; for interactive DBA work on a bastion or jumphost, HTTPIe is the better experience.

info

Exceptions Where curl Still Wins

- Scripts and automation. curl is on every Linux image; HTTPIe is not. Every shell script that runs across heterogeneous hosts should use curl.
- Container probes and healthchecks. curl is ~150 KB statically linked; HTTPIe pulls in CPython and requests. Image size matters.
- Non-HTTP protocols. curl speaks FTP, SFTP, SMTP, IMAP, LDAP, and a dozen others. HTTPIe is HTTP-only.
- Maximum performance. For benchmark loads and parallel transfers, curl with `--parallel` or a dedicated tool like `vegeta` or `wrk` is far faster than HTTPIe.
- Strict compatibility. Every example in every database vendor's documentation uses curl. Pasting the documented command unchanged is sometimes the right answer.

Comparable Tools

- `xh` — a Rust reimplementation of HTTPIe. Single static binary, dramatically faster startup, same syntax. Worth installing on bastions where HTTPIe's ~400 ms cold-start matters.
- `curlie` — a thin wrapper that gives curl the HTTPIe syntax while keeping curl's behavior and performance. Good middle ground.
- Postman / Insomnia — graphical clients. Useful for exploration, less useful for an SSH session into a database host.
- `wget` — discussed in the next chapter; download-oriented, not API-oriented.

Conclusion

Recommendation — keep curl in scripts and container images; install HTTPIe (or xh) on bastions and developer laptops for interactive API work against database admin endpoints,



PostgREST, metrics scrapers, and cloud database control planes. The two tools coexist; they are not substitutes in the strict sense.

ok

Chapter 21 · wget → curl -O / aria2c

The Problem

A DBA downloads things at almost every stage of a database's lifecycle: vendor binaries during installation, patches during upgrades, backup files during recovery, archive logs during point-in-time restores, and exported data during migrations. wget, written in 1996, has been the default Linux download tool for two decades. It works, it retries, it can resume — and for many DBAs it is the only download command they have ever typed. The problem is not that wget is broken; the problem is that on minimal modern images and inside containers it is often not installed, and the same machines almost always have curl.

The Modern Solution

There are two modern paths. For ad-hoc downloads where curl is already present, curl -O (capital O) downloads a single file to the current directory using the remote filename; curl -L -O follows redirects, which most CDN-served binaries require. For high-throughput downloads where the bottleneck is per-connection bandwidth — large database dumps, OS image downloads, multi-gigabyte archive log restores — aria2c opens multiple parallel connections to the same file and reassembles them, often achieving 2 to 5 times the throughput of a single-connection downloader.

Side-by-Side Comparison

Task	wget	curl / aria2c
Simple download	wget https://host/file.dump	curl -O https://host/file.dump
Follow redirects	wget (default)	curl -L -O https://host/file.dump
Resume partial download	wget -c https://host/file.dump	curl -C - -O https://host/file.dump
Rate-limit	wget --limit-rate=10M url	curl --limit-rate 10M -O url
Mirror entire directory	wget -r -np -nH https://host/dir/	wget still wins here
Parallel multi-connection	not supported	aria2c -x 8 -s 8 https://host/file.dump
Resume with parallelism	not supported	aria2c -c -x 8 https://host/file.dump

Real-World DBA Scenario

A MySQL DBA is restoring a 220 GB logical dump from S3 to a recovery host in a different region. The link is 1 Gbps but per-connection throttling at the egress proxy caps a single TCP stream at roughly 80 Mbps. With wget the download would take just over six hours. The DBA switches to aria2c with eight parallel connections:

```
$ aria2c -x 8 -s 8 -c --file-allocation=falloc \
```

```
https://backup-bucket.s3.us-east-1.amazonaws.com/dumps/2026-05-20-  
full.sql.zst
```

Throughput climbs to roughly 620 Mbps and the download completes in 50 minutes. The `-c` flag enables resume-on-failure; the `--file-allocation=falloc` flag pre-allocates the file with `fallocate(2)`, avoiding fragmentation on ext4. The same DBA was previously using `wget` for this restore weekly; switching to `aria2c` reclaimed roughly five hours of recovery time per restore drill.

Warning — parallel download tools shift the bottleneck. If the upstream is S3 the parallelism helps; if the upstream is a small on-prem web server, eight parallel connections can saturate the server and trigger rate-limiting or worse, get the source DBA paged. Always coordinate with the source host owner before turning up parallelism.

warn

Exceptions Where wget Still Wins

- Recursive mirroring. `wget -r -np -nH` for cloning a static website or a directory of archive logs is genuinely better than the `curl` or `aria2c` equivalent.
- Background daemon downloads. `wget -b` detaches and writes to `wget-log`; convenient for long downloads on a session you intend to disconnect.
- Robust defaults. `wget` retries automatically; `curl` does not retry by default and needs `--retry 5` to match.
- Existing runbooks. If your team's runbook says `wget`, do not introduce a different tool during an incident. Standardize during a calm change window, not at 03:00.

Comparable Tools

- `axel` — older parallel download tool, simpler than `aria2c`, fewer features but lower install footprint.
- `rclone` — for object-storage downloads (S3, GCS, Azure Blob, B2). For database backup retrieval directly from a bucket, `rclone copy` is almost always faster and more correct than `curl` on a presigned URL.
- `s5cmd` — extremely fast Go-based S3 client. For PostgreSQL WAL archive restores from S3, `s5cmd cp` regularly outperforms `aws s3 cp` by 5–10x.
- AWS CLI / `gcloud` / `az` — vendor-specific tools that handle authentication, paging, and integrity automatically. For cloud-native databases, prefer these over generic downloaders.

Conclusion

Recommendation — for one-off downloads use whichever of `curl` or `wget` is already on the host. For large database backup or restore traffic that fits on a single URL, install `aria2c` and



use 4–8 parallel connections after coordinating with the source. For object-storage backed backups, use the vendor CLI or rclone rather than any HTTP downloader.

ok

Chapter 22 · scp → rsync

The Problem

scp has been the default DBA-to-DBA file copy command since the late 1990s. It is simple, it uses the same ssh credentials that already work, and the syntax is memorable: `scp source user@host:dest`. But scp has two fatal characteristics for a DBA who moves large database files. First, it has no resume — a transfer interrupted at 95% restarts from zero. Second, it always re-copies the entire file regardless of what already exists at the destination; there is no notion of "copy only what has changed." For a 500 GB PostgreSQL base backup, both characteristics translate directly into recovery time.

OpenSSH 9.0, released in April 2022, also changed scp's underlying protocol from the legacy SCP/RCP protocol to SFTP by default — a security improvement, but one that broke a number of long-standing scp behaviors and made the case for moving to rsync even stronger.

The Modern Solution

rsync, written by Andrew Tridgell in 1996, performs delta-transfer: it compares source and destination and transfers only the blocks that differ. For database files specifically — where a multi-gigabyte tablespace file may have only a few megabytes of changed blocks since the last copy — the savings are dramatic. rsync resumes interrupted transfers automatically, supports compression on the wire, can run over ssh with no daemon required, and is on every Linux distribution by default. For the DBA, rsync is what scp should have been.

Side-by-Side Comparison

Task	scp	rsync
Copy single file	<code>scp file.dump host:/dest/</code>	<code>rsync -a file.dump host:/dest/</code>
Copy directory	<code>scp -r dir/ host:/dest/</code>	<code>rsync -a dir/ host:/dest/</code>
With compression	<code>scp -C file.dump host:/dest/</code>	<code>rsync -az file.dump host:/dest/</code>
Show progress	scp (basic counter)	<code>rsync -ah --progress file host:/dest/</code>
Resume after failure	not supported	<code>rsync --partial --append-verify</code>
Sync only changed blocks	not supported	<code>rsync -a</code> (default)
Dry-run preview	not supported	<code>rsync -avn src/ host:dest/</code>
Bandwidth limit	<code>scp -l 10000</code>	<code>rsync --bwlimit=10M</code>
Preserve hardlinks	not supported	<code>rsync -aH</code>

Real-World DBA Scenario

A PostgreSQL DBA maintains a warm-standby host that lags the primary by 30 minutes and is refreshed nightly from a base backup. The base backup is 480 GB. Using `scp` the nightly refresh takes roughly 95 minutes over the 1 Gbps internal link and re-transfers every byte. Switching to `rsync` with the standard flag set:

```
$ rsync -azh --partial --inplace --info=progress2 \  
    /var/lib/postgresql/16/main/ \  
    standby:/var/lib/postgresql/16/main/
```

After the first full sync, subsequent nightly runs transfer only the blocks that changed during the day — typically 12 to 25 GB out of 480 GB — and complete in 4 to 9 minutes. The `--inplace` flag instructs `rsync` to write changed blocks directly into the destination file rather than to a temporary file, which is the right choice for large database files because it avoids briefly doubling the disk usage on the standby. The `--partial` flag preserves partial transfers across interruptions; the next `rsync` resumes from where the previous one stopped.

Warning — `rsync --inplace` is unsafe for files that other processes read while they are being modified. Never `rsync --inplace` into a running database's data directory. The standby example above works only because PostgreSQL is stopped on the standby during the refresh. For an online sync, omit `--inplace`.

warn

OpenSSH 9.0 note — since OpenSSH 9.0 (April 2022), `scp` uses SFTP under the hood. Many existing `scp` invocations still work, but server-side glob expansion no longer behaves the same way. If a long-standing `scp` command starts failing after a host upgrade, this protocol change is the most likely cause; pass `-O` to force the legacy protocol as a temporary workaround.

info

Exceptions Where `scp` Still Wins

- Truly one-shot small files. For a 20 KB config file, `scp` is one less character to type and the delta-transfer overhead of `rsync` is irrelevant.
- Environments without `rsync`. `rsync` must be installed on both sides; in extremely minimal container or appliance environments, only `scp` may be available.
- Strict change-management. Some compliance regimes pin a specific transfer protocol; `scp` may be the audited tool.
- Pre-shared-key automation where the receiving side cannot run `rsyncd` or `sshd`-launched `rsync`.

Comparable Tools

- `sftp` — interactive secure file transfer; useful for exploratory file moves, less useful for automation.



- rclone — for transfers to or from object storage; rclone speaks S3, GCS, Azure Blob, Backblaze B2, and roughly 70 other backends.
- BSCP / GridFTP / Aspera — high-performance parallel-stream transfer tools for very long fat networks (intercontinental, 10 Gbps+). For most DBA needs, rsync over a well-tuned ssh is sufficient.
- pg_basebackup — for PostgreSQL specifically, this is the correct tool for an initial standby seed; do not use rsync as a substitute for the documented backup tool.
- Percona XtraBackup / mariabackup — for MySQL/MariaDB, the equivalent vendor-supported backup tools.

Conclusion

Recommendation — make `rsync -azh --partial --info=progress2` the team-standard transfer command. Reserve `scp` for files small enough that resume is not a concern. For initial database seeds, always prefer the vendor's documented backup tool (`pg_basebackup`, `XtraBackup`) over `rsync`; reserve `rsync` for subsequent refreshes and for non-database file movement.

ok

Chapter 23 · gzip / tar -czf → zstd / tar --zstd

The Problem

Compression is the single highest-leverage optimization in the DBA's toolkit. Every backup written, every WAL segment archived, every dump shipped between regions either crosses a compression boundary or wastes storage and bandwidth that the business pays for. For thirty years gzip has been the default. It is on every Linux host, every backup tool supports it, and the .gz extension is universally understood. The problem is that gzip's compression ratio and speed both date from 1992 hardware, and database backup workloads have changed beyond recognition since.

The Modern Solution

Zstandard (zstd), released by Facebook in 2016 and now maintained as an open standard with native kernel support, dominates gzip on essentially every axis that matters for database workloads. At its default compression level (zstd -3) it produces a smaller output than gzip -6 while compressing roughly three to five times faster, and decompresses six to ten times faster. At its low end (zstd -1) it can saturate a 1 Gbps link from CPU alone while still compressing better than gzip. At its high end (zstd -19 with --long) it approaches xz's compression ratio at a fraction of the CPU cost. For database backups, which are CPU-bound on a single compressor thread and which spend most of their lifetime as stored bytes waiting to be restored, zstd is a strict improvement.

PostgreSQL 15 added native zstd support in pg_dump and pg_basebackup. MySQL 8.0.18+ supports zstd in mysqlpump and as a connection compression protocol. MongoDB has supported zstd network compression since 4.2. The database vendors have already migrated; the DBAs frequently have not.

Side-by-Side Compression Comparison

The table below summarizes typical results on a 10 GB PostgreSQL dump on a modern x86_64 server. Specific numbers vary with data shape; the ratios are stable across most database backup workloads.

Tool	Compress time	Decompress time	Output size	Notes
gzip -6 (default)	≈ 240 s	≈ 35 s	≈ 2.1 GB	Baseline
gzip -9	≈ 480 s	≈ 33 s	≈ 2.0 GB	Marginal gain, large cost
pigz -6 (parallel gzip)	≈ 35 s	≈ 30 s	≈ 2.1 GB	Parallel gzip; same ratio
zstd -3 (default)	≈ 55 s	≈ 5 s	≈ 1.9 GB	Faster + smaller than gzip
zstd -10	≈ 130 s	≈ 5 s	≈ 1.6 GB	Sweet spot for backups
zstd -19 --long	≈ 1100 s	≈ 6 s	≈ 1.3 GB	Archival-grade
xz -6	≈ 1400 s	≈ 70 s	≈ 1.4 GB	Slow compress + decompress

Side-by-Side Command Comparison

Task	gzip-era	zstd-era
Compress single file	gzip file.dump	zstd file.dump
Decompress	gunzip file.dump.gz	unzstd file.dump.zst
Compress in place, keep original	gzip -k file.dump	zstd -k file.dump
Highest compression	gzip -9 file.dump	zstd -19 --long file.dump
Stream compress	command gzip > out.gz	command zstd > out.zst
tar + compress	tar -czf archive.tar.gz dir/	tar --zstd -cf archive.tar.zst dir/
Parallel compression	pigz file.dump	zstd -T0 file.dump
List archive contents	gzip -l file.gz	zstd -l file.zst

Real-World DBA Scenario

A PostgreSQL platform team runs nightly logical dumps of a 380 GB OLTP database. The historic command, embedded in a 2017 backup script, is `pg_dump -Fc | gzip > backup.dump.gz`. The dump produces 78 GB of compressed output and takes 2 hours 40 minutes; gzip is single-threaded and pinning one CPU core is the bottleneck. The DBA migrates the script to PostgreSQL 16's native zstd support:

```
# Old
pg_dump -Fc mydb | gzip -6 > /backup/mydb-$(date +%F).dump.gz

# New (PostgreSQL 15+)
pg_dump -Fc -Z zstd:10 mydb -f /backup/mydb-$(date +%F).dump.zst

# Or with parallel compression
pg_dump -Fc -Z zstd:10 -j 4 mydb -f /backup/mydb-$(date +%F).dump.zst
```

Compressed size drops from 78 GB to 61 GB; dump time drops from 2:40 to 0:38. The S3 transfer cost falls in proportion, and the nightly window — formerly the long pole in the maintenance window — is no longer a constraint. The restore-time improvement is even larger: a streaming restore that previously spent 22 minutes decompressing now spends roughly 4 minutes.

Compatibility note — zstd-compressed backups require zstd on the restore host. For modern Linux distributions this is a single package; for older RHEL 7 hosts and ancient containers, install from EPEL or compile. Never write zstd backups to long-term archival storage without confirming that the recovery environment will have a zstd decompressor available decades from now. For genuinely archival material, plain tar plus a documented external decompressor remains the safer choice.

warn

Exceptions Where gzip Still Wins

- Web compression and protocols that require gzip. HTTP Accept-Encoding negotiation, log shippers that only understand .gz, and old vendor tools that expect gzip — these have long compatibility tails.
- Maximum portability. Every Unix-like system since 1992 has gzip. A backup written as .gz can be restored on a system the DBA has never heard of. zstd, while widely available, is not yet universal on legacy AIX, Solaris, or HP-UX hosts that still host production databases in some industries.
- Streaming over a very slow CPU. On embedded hardware or extremely low-end VMs, gzip's CPU profile may still be preferable to zstd's higher per-byte cost at high compression levels.
- Existing toolchain integration. A monitoring or audit tool that scans .gz files and parses them in-process may not support .zst; the cost of replacing the tool can exceed the savings.
- Cryptographic verification chains. If existing detached signatures and checksums are computed over .gz files, changing the compression breaks the chain; this is usually a calendar decision, not a technical one.

Comparable Tools

- pigz — parallel gzip. Drop-in replacement for gzip that uses all cores. If migrating to zstd is blocked for compatibility reasons, install pigz and recover the parallelism win without changing the file format.
- lz4 — even faster than zstd, weaker compression ratio. Useful when the bottleneck is decompression speed and storage is cheap (e.g., in-memory column stores, hot tier backups).
- xz — best compression ratio of the mainstream compressors but extremely slow on both compress and decompress. Reserved for cold archival.
- bzip2 — historic middle ground between gzip and xz; superseded on every axis by zstd. No new code should adopt bzip2.
- Built-in database compression — PostgreSQL TOAST with lz4 (15+), MySQL InnoDB page compression, MongoDB WiredTiger zstd. These operate on stored data and complement, rather than replace, backup compression.

Conclusion

Recommendation — standardize on zstd for new database backup and shipping pipelines, with -10 as the default level for backups (good ratio, modest CPU) and -3 for in-band streaming where CPU is the bottleneck. Use -T0 for parallel compression on multi-core backup hosts. Migrate from gzip to zstd opportunistically as backup scripts are touched; do not undertake a forklift migration of historical .gz archives unless the storage savings clearly



justify the operational risk. Keep gzip available for compatibility with external systems that require it.

ok



Part VII • Migration Strategy

The preceding chapters compared individual commands. This part addresses the harder problem: how a DBA team that has used a particular set of tools for a decade actually changes its habits without breaking production during the transition. Tool migration is a sociotechnical exercise; the technical part is the easy half.

Chapter 24 · Adoption Strategy for DBA Teams

The Problem

A book that proposes fifty command changes implicitly proposes a fifty-step migration. Executed naively, the migration introduces fifty new failure modes simultaneously, breaks every runbook that currently works, and trains the team on commands that exist on the upgraded hosts but not yet on the legacy hosts. The result is a regression — slower response time during incidents, junior engineers paralyzed by ambiguity, and senior engineers quietly continuing to use the legacy commands while the new ones gather dust. The migration strategy matters more than any individual tool choice.

A Phased Rollout

The migration approach that consistently works in production database teams is a five-phase rollout over six to twelve months. Each phase has a defined exit criterion; the team does not advance until the criterion is met.

Phase 1 — Personal tooling (weeks 1–4)

Each DBA installs the modern toolset on their personal workstation, laptop, and bastion host accounts. No production hosts are touched. The goal is purely individual familiarity: the DBA should be able to type `ip addr`, `ss -tulnp`, `journalctl -u postgresql -p err -S '1 hour ago'`, and `rg deadlock` from muscle memory before any team-wide change is proposed. Exit criterion: every DBA on the team can complete a written list of fifteen common diagnostic tasks using only modern commands.

Phase 2 — Non-production hosts (weeks 5–10)

Modern tools are installed via configuration management (Ansible, Puppet, Salt) on development, staging, and QA database hosts. Existing legacy commands remain installed alongside; nothing is removed. The team's internal runbook is updated to add a "modern equivalent" column next to each legacy command, but the legacy command remains the documented primary. Exit criterion: thirty consecutive days with no operational incident attributable to the new tools, and at least three documented uses of the modern tools during routine non-production work.

Phase 3 — Production read-only (weeks 11–18)

Modern tools are deployed to production hosts for diagnostic use only. Runbooks are updated to make the modern commands the documented primary for read-only operations: showing routes, listening sockets, processes, disk usage, and logs. Write or modify operations — bringing interfaces up or down, restarting services, killing processes — continue to use the documented legacy commands until a senior DBA has explicitly signed off on the modern equivalent. Exit criterion: ninety consecutive days with no production incident related to read-only diagnostic commands, and runbook updates merged.

Phase 4 — Production write operations (weeks 19–30)

Modern tools take over write operations one category at a time: services first (systemctl), then scheduling (systemd timers replacing cron jobs), then network (ip replacing ifconfig), then transfer (rsync replacing scp in backup scripts). Each category waits for the prior one to stabilize. Exit criterion: each category runs for sixty days without a tool-related incident before the next is started.

Phase 5 — Decommission (weeks 31+)

Legacy commands are removed from new host builds. Existing hosts retain them indefinitely; there is no operational benefit to forcibly removing a working tool. New runbooks no longer reference legacy commands. Onboarding documentation for new DBAs teaches the modern commands as primary, with a single appendix noting the legacy equivalents for historical reference.

Alias Safety — A Common Anti-Pattern

A frequent shortcut is to alias the legacy commands to the modern ones — for example, `alias ls='eza'` or `alias cat='bat'`. This appears clever and works on the DBA's personal shell. It is a production liability. Aliases live in a single user's shell configuration; they do not survive when a script invokes the command, when `sudo` runs the command as root, when a cron job executes, when a CI pipeline runs the command, or when a different DBA logs in to the same host during an incident. The result is two parallel worlds — the aliased shell and reality — that diverge unpredictably.

Recommendation — never alias a system command to its modern replacement. Type the modern command explicitly. The cost is a few keystrokes; the benefit is that what you see in your shell is what every other shell on the host also sees.

warn

Runbook Migration

Runbooks are the institutional memory of an operations team. A runbook that contains `netstat -tulnp` continues to be correct on a host where `netstat` is installed and incorrect on a host where it is not, and the latter category is growing. The runbook migration follows a simple discipline:

1. Inventory every runbook command. A simple `grep` across the runbook repository produces a list of every legacy command in active documentation.
2. Pair each legacy command with its modern equivalent and document both in the same runbook step, with the modern command listed first.
3. Add a one-line explanation of why the modern command is preferred, so a junior DBA reading the runbook understands the rationale and does not silently revert.
4. Run a quarterly review of the runbook repository; any runbook step that has not been executed in twelve months is reviewed by the on-call lead and either deleted or revalidated.

Training and Onboarding



A migration without training fails. The cheapest and most effective format is a one-hour weekly "command pair of the week" session for the duration of phases 2 through 4. Each session covers two or three command pairs: the legacy command, the modern equivalent, the differences that matter, and a live demonstration of the modern command against a non-production host. The recording is archived and becomes part of new-hire onboarding. The total training investment is roughly thirty hours per DBA over six months — recovered many times over by faster diagnostic time during the first significant incident after the migration completes.

Measuring Success

The migration is not finished when the modern tools are installed; it is finished when DBAs reach for them by default during an incident. Two metrics matter:

- Time-to-first-useful-output during incident reviews. Compare the time from "engineer logs in" to "first relevant diagnostic command produces output" before and after migration. A drop of 20–40% is typical and reflects faster commands plus better defaults.
- Runbook drift rate. Count the number of runbook steps that reference commands not installed on the host they run against. The migration is complete when this number is zero for production hosts.

Chapter 25 · Building a Modern DBA Toolkit

The Problem

The preceding chapters introduced roughly thirty tools across networking, services, processes, files, transfer, compression, logging, and scheduling. Installing them ad hoc on each host as the need arises produces an inconsistent fleet: tool A is on the bastion but not on the database hosts, tool B is on RHEL hosts but not on Ubuntu, version skew breeds incident-time confusion. A documented, packaged toolkit is the only practical answer.

The Recommended Toolkit

The following install list represents a practical default for a database administration team running PostgreSQL, MySQL, or MongoDB on modern Linux. The list is intentionally short; every tool earns its place by appearing in a documented runbook or training session.

Category	Tool	Why included
Networking	iproute2 (ip, ss)	Replaces ifconfig, route, netstat; mandatory
DNS	bind-utils / dnsutils (dig)	Authoritative DNS diagnostic
Connectivity	mtr, nc (ncat or openssh-netcat)	Latency + TCP port testing
Services	systemd (systemctl, journalctl)	On every modern distribution; learn it deeply
Scheduling	systemd timers	Replaces cron for new jobs
Processes	htop, btop	htop is universal; btop for richer view
Disk	df, du, ncdu	Modern df/du; ncdu for interactive cleanup
File listing	eza (not exa)	Maintained successor to deprecated exa
File viewing	bat (or batcat on Debian/Ubuntu)	Syntax-highlighted cat/less
File search	fd, ripgrep (rg)	Faster, simpler than find and grep
Text processing	miller (mlr), sd (optional)	CSV/JSON/awk replacement; sd is optional
HTTP	curl (always), httpie or xh	curl for scripts, httpie for interactive
Transfer	rsync, aria2c, rclone	rsync for hosts, aria2c for large URLs, rclone for object stores
Compression	zstd, pigz	zstd for new pipelines, pigz for gzip parallelism
JSON	jq (always), yq	Mandatory for cloud database APIs

Package Installation by Distribution

The commands below install the full toolkit on each major distribution. Run as root or via sudo. Some packages live in non-default repositories (EPEL on RHEL, third-party on older Ubuntu); the chapter on installation in Appendix B covers those cases.

Debian and Ubuntu (22.04 LTS and newer)

```
sudo apt update && sudo apt install -y \  
  iproute2 dnsutils mtr netcat-openbsd \  
  systemd htop duf ncdu \  
  eza bat fd-find ripgrep miller \  
  curl httpie rsync aria2 rclone \  
  zstd pigz jq  
  
# On Debian/Ubuntu, 'bat' may be installed as 'batcat' to avoid  
# a name collision with an old utility. Create a personal symlink:  
mkdir -p ~/.local/bin  
ln -sf /usr/bin/batcat ~/.local/bin/bat  
ln -sf /usr/bin/fdfind ~/.local/bin/fd
```

RHEL, Rocky, AlmaLinux 9

```
sudo dnf install -y epel-release  
sudo dnf install -y \  
  iproute bind-utils mtr nmap-ncat \  
  systemd htop ncdu \  
  bat fd-find ripgrep \  
  curl rsync aria2 \  
  zstd pigz jq  
  
# eza, duf, dust, miller, httpie are not in EPEL on RHEL 9.  
# Install via cargo (Rust), pip (Python), or download a static binary  
# from the project release page. See Appendix B for details.
```

Container Images

For Alpine-based containers and minimal Debian containers used for utility pods and operator sidecars, install only the read-only diagnostic subset. Avoid installing modify-state tools in containers; let the operator perform those actions from the bastion.

```
# Alpine sidecar – diagnostic-only image  
RUN apk add --no-cache \  
  iproute2 bind-tools mtr \  
  htop \  
  ripgrep jq curl
```

Version Pinning and Reproducibility

Tool behavior changes across versions. ripgrep added --search-zip in v0.10; zstd added the --long flag in v1.3.2; systemd's journalctl output format changed multiple times across the v240 to v255 release



series. A team that runs an Ansible role to install the toolkit should pin to a known-good version per environment and bump those versions deliberately, not opportunistically. Version pinning is also what allows a runbook command to be valid across the fleet — if `rg --search-zip` is documented, every host must have a `ripgrep` that supports it.

Conclusion

Recommendation — codify the toolkit list above as a configuration-management role, version-pinned per environment, and applied to every host the team operates. Treat the toolkit as production software: review it quarterly, document deprecations as they happen, and ensure that every command in a runbook is provided by a tool in the toolkit. The toolkit is the contract between the team's runbooks and reality.

ok



Appendix A · Quick Reference Cheat Sheet

The complete table of fifty traditional commands and their modern equivalents, organized by functional category. This is the single page to print and pin above the desk.

A.1 Networking — Interfaces and Routing

#	Traditional	Modern	Purpose
1	ifconfig	ip addr	Show network interfaces and IP addresses
2	ifconfig eth0 up	ip link set eth0 up	Enable network interface
3	ifconfig eth0 down	ip link set eth0 down	Disable network interface
4	route -n	ip route	Show routing table
5	route add	ip route add	Add a route
6	netstat -tulnp	ss -tulnp	Show listening ports
7	netstat -rn	ip route	Show routing table
8	netstat -i	ip -s link	Show interface statistics
9	arp -a	ip neigh	Show ARP / neighbor table
10	arp -d	ip neigh del	Delete ARP entry
11	brctl show	bridge link	Show Linux bridge info
12	brctl addbr br0	ip link add br0 type bridge	Create bridge
13	brctl addif br0 eth0	ip link set eth0 master br0	Add interface to bridge
14	iwconfig	iw	Wireless network configuration

A.2 DNS and Connectivity

#	Traditional	Modern	Purpose
15	nslookup	dig	DNS lookup
16	nslookup domain.com	dig domain.com	Query DNS records
17	host domain.com	dig domain.com	DNS query
18	tracert	tracert / mtr	Trace network path
19	ping	ping -c 4	Bounded connectivity test
20	telnet host port	nc -vz host port	Test TCP port connectivity

#	Traditional	Modern	Purpose
21	ftp	sftp	Secure file transfer

A.3 Services and Boot

#	Traditional	Modern	Purpose
22	rc.local	systemd .service unit	Startup scripts
23	service nginx start	systemctl start nginx	Start service
24	service nginx stop	systemctl stop nginx	Stop service
25	service nginx status	systemctl status nginx	Check service status
26	chkconfig nginx on	systemctl enable nginx	Enable service at boot
27	chkconfig nginx off	systemctl disable nginx	Disable service at boot
28	/etc/init.d/nginx restart	systemctl restart nginx	Restart service

A.4 Processes and Resource Monitoring

#	Traditional	Modern	Purpose
29	ps aux	ps -ef / pgrep	Show processes
30	ps aux grep postgres	pgrep -a postgres	Find process by name
31	top	htop	Interactive process monitor
32	top	btop	Modern system monitor
33	free -m	free -h	Human-readable memory usage
34	df -h	duf	Modern disk usage viewer
35	du -sh *	dust	Modern disk usage analyzer

A.5 Files, Search, and Text Processing

#	Traditional	Modern	Purpose
36	ls	eza	Modern file listing
37	ls -la	eza -la	Detailed modern listing
38	cat file.txt	bat file.txt	View file with syntax highlighting
39	less file.txt	bat file.txt / less	View file content
40	find / -name file	fd file	Modern file search

#	Traditional	Modern	Purpose
41	grep "text" file	rg "text" file	Fast text search
42	sed	sd	Simpler find-and-replace
43	awk	mlr	Structured data processing

A.6 HTTP, Transfer, and Compression

#	Traditional	Modern	Purpose
44	curl	httplib / xh	User-friendly HTTP client
45	wget	curl -O / aria2c	Download files
46	scp	rsync -avz	File copy with resume / delta-sync
47	tar -czf	tar --zstd -cf	Faster, smaller archive compression
48	gzip	zstd	Modern compression

A.7 Scheduling and Logging

#	Traditional	Modern	Purpose
49	crontab	systemd timer + .service	Scheduled jobs
50	tail /var/log/*	journalctl	View systemd-managed logs

Appendix B • Installation Guide

Detailed installation commands for each modern tool referenced in the book, organized by Linux distribution and by tool. Where a tool is not available in the distribution's default repositories, alternative installation methods are noted.

B.1 Universal — Tools Available in Every Mainstream Distribution

The following tools ship in the default repositories of Debian 11+, Ubuntu 20.04+, RHEL 8+ (with EPEL for some), Fedora 35+, and openSUSE Leap 15.4+. A single apt or dnf invocation installs the full set.

Tool	Debian/Ubuntu package	RHEL/Fedora package
ip, ss	iproute2	iproute
dig	dnstools	bind-utils
mtr	mtr-tiny	mtr
nc	netcat-openbsd	nmap-ncat
systemctl, journalctl	systemd	systemd (preinstalled)
htop	htop	htop (EPEL on RHEL 8)
rsync	rsync	rsync
zstd	zstd	zstd
pigz	pigz	pigz
jq	jq	jq
curl	curl	curl
aria2c	aria2	aria2 (EPEL)
ripgrep	ripgrep (20.04+)	ripgrep (EPEL)
fd	fd-find (installed as fdfind)	fd-find
bat	bat (installed as batcat)	bat

B.2 Tools Requiring Alternative Installation

eza — modern ls

eza is the actively maintained fork of exa, which was archived in 2023. On Ubuntu 24.04 and newer eza is in the default repositories. On older Ubuntu and RHEL it must be installed from the project's GitHub releases or via cargo.

```
# Ubuntu 24.04+ / Debian 13+
```



```
sudo apt install eza

# Older Ubuntu / Debian – install from project repo
sudo mkdir -p /etc/apt/keyrings
wget -qO- https://raw.githubusercontent.com/eza-community/eza/main/deb.asc \
| sudo gpg --dearmor -o /etc/apt/keyrings/gierens.gpg
echo 'deb [signed-by=/etc/apt/keyrings/gierens.gpg] http://deb.gierens.de stable
main' \
| sudo tee /etc/apt/sources.list.d/gierens.list
sudo apt update && sudo apt install eza

# RHEL / Rocky / Alma – via cargo (requires Rust toolchain)
cargo install eza
```

duf and dust — modern df and du

```
# Debian/Ubuntu: in default repos from 22.04 onward
sudo apt install duf

# dust is not in apt – install from GitHub releases:
ARCH=$(uname -m)
VER=1.0.0
wget https://github.com/bootandy/dust/releases/download/v${VER}/dust-v${VER}-
${ARCH}-unknown-linux-gnu.tar.gz
tar xzf dust-v${VER}-${ARCH}-unknown-linux-gnu.tar.gz
sudo install dust-v${VER}-${ARCH}-unknown-linux-gnu/dust /usr/local/bin/
```

httpie and xh — modern HTTP clients

```
# httpie – Python-based, in default repos
sudo apt install httpie      # Debian/Ubuntu
sudo dnf install httpie      # Fedora (EPEL on RHEL)

# xh – Rust reimplementation, much faster startup
cargo install xh
# or download a static binary from https://github.com/ducaale/xh/releases
```

btop — modern top

```
sudo apt install btop      # Debian 12+ / Ubuntu 22.04+
sudo dnf install btop      # Fedora / RHEL 9+ (EPEL)
```

miller (mlr) — structured data awk

```
sudo apt install miller      # Debian/Ubuntu
sudo dnf install miller      # Fedora / RHEL (EPEL)
```

sd — simpler sed

```
# Not in most default repos; install via cargo
```



```
cargo install sd
# or download from https://github.com/chmln/sd/releases
```

rclone — object storage transfer

```
# Use the upstream installer for the most current version
curl https://rclone.org/install.sh | sudo bash
```

B.3 Verification After Installation

After installing the toolkit, run the following one-line verification to confirm every binary is reachable and produces a version string. The output should be a single line per tool with no "command not found" errors.

```
for t in ip ss dig mtr nc systemctl journalctl htop btop \
    duf dust eza bat fd rg mlr curl httpie aria2c \
    rsync rclone zstd pigz jq; do
    printf '%-12s ' "$t"
    command -v "$t" >/dev/null 2>&1 \
        && "$t" --version 2>/dev/null | head -1 \
        || echo 'NOT INSTALLED'
done
```

References

Sources consulted during the preparation of this book. All commands and behavioral claims were verified against the cited primary documentation as of May 2026; tool versions and behavior may change after publication.

- Linux Kernel Organization. (2026). iproute2: utilities for controlling TCP/IP networking and traffic control in Linux. *Linux Kernel Archives*. <https://www.kernel.org/pub/linux/utils/net/iproute2/>
- Poettering, L., Sievers, K., et al. (2026). systemd: System and Service Manager — manual pages. *freedesktop.org*. <https://www.freedesktop.org/software/systemd/man/>
- Red Hat, Inc. (2024). Networking Guide: Deprecated network interface naming and the migration to iproute2. *Red Hat Customer Portal*. https://access.redhat.com/documentation/en-us/red_hat_enterprise_linux/9/html/configuring_and_managing_networking/
- eza-community. (2026). eza: A modern alternative to ls — README and migration notes from exa. *GitHub*. <https://github.com/eza-community/eza>
- Gallant, A. (2026). ripgrep (rg): Recursively searches directories for a regex pattern — User Guide. *GitHub*. <https://github.com/BurntSushi/ripgrep>
- Kerr, J. (2026). Miller: Like awk, sed, cut, join, and sort for name-indexed data such as CSV, TSV, and tabular JSON. *Miller Project Documentation*. <https://miller.readthedocs.io/>
- Collet, Y., & Kucherawy, M. (2021). Zstandard Compression and the application/zstd Media Type (RFC 8878). *Internet Engineering Task Force*. <https://www.rfc-editor.org/rfc/rfc8878>
- Facebook, Inc. (2026). Zstandard — Real-time data compression algorithm. *Zstandard Project*. <https://facebook.github.io/zstd/>
- OpenBSD Project. (2022). OpenSSH 9.0 Release Notes: SCP now uses SFTP protocol by default. *OpenSSH*. <https://www.openssh.com/txt/release-9.0>
- Tridgell, A., & Mackerras, P. (2026). rsync — a fast, versatile, remote (and local) file-copying tool. *rsync Project*. <https://rsync.samba.org/documentation.html>
- Stenberg, D. (2026). curl: command line tool and library for transferring data with URLs — Manual. *curl Project*. <https://curl.se/docs/manual.html>
- PostgreSQL Global Development Group. (2026). PostgreSQL 16 Documentation — Chapter 26: Backup and Restore. *PostgreSQL Documentation*. <https://www.postgresql.org/docs/16/backup.html>
- MySQL AB / Oracle Corporation. (2026). MySQL 8.0 Reference Manual — Connection Compression with zstd. *MySQL Documentation*. <https://dev.mysql.com/doc/refman/8.0/en/connection-compression-control.html>
- MongoDB, Inc. (2026). MongoDB Manual — Network Compression. *MongoDB Documentation*. <https://www.mongodb.com/docs/manual/administration/network-compression/>
- Debian Project. (2024). net-tools package: Status and deprecation notice. *Debian Package Tracker*. <https://tracker.debian.org/pkg/net-tools>



-
- Bell, F., et al. (2024). HTTPie: command-line HTTP and API testing client for humans — Documentation. *HTTPie Project*. <https://httpie.io/docs>
- aria2 Project. (2026). aria2 Manual — Multi-source download with HTTP(S), FTP, BitTorrent, and Metalink. *aria2*. <https://aria2.github.io/manual/en/html/>

About TheDBAdmin and the Author

About TheDBAdmin.com

TheDBAdmin.com is a training and content platform focused on the practicing database administrator. Its core mission is to bridge the widening gap between traditional database operations and the modern infrastructure, automation, and AI-assisted workflows that now surround the database tier. The platform publishes long-form practitioner books, hands-on courses, and corporate training programs covering PostgreSQL, MySQL, MongoDB, prompt engineering, generative AI for DBAs, and the emerging discipline of agentic AI in database operations.

Content is structured around the principle that every concept should begin with the problem it solves, proceed through the solution and the system design, and conclude with the real-world scenarios, trade-offs, and exceptions a working DBA will actually encounter. The platform's audience spans Fortune 500 enterprise teams in India and the United States as well as individual database professionals worldwide.

About the Author

Versha is the founder of TheDBAdmin.com and an agentic AI corporate trainer with a background spanning two decades of database administration, generative AI, and software engineering. She holds a degree in Computer Engineering (2003–2007) and built one of her earliest projects — a PHP-based chatbot — as her undergraduate capstone, an experience she now connects directly to her current work in large language models and agentic systems.

Her recent published work includes a 57-page practitioner book on agentic AI for DBAs, a 10-module corporate curriculum on production agentic AI systems covering LangGraph, MCP, A2A protocols, CrewAI, RAG, pgvector, human-in-the-loop, observability, and deployment, and a series of field guides on PostgreSQL architecture, query tuning, and shell-scripting automation for DBAs. She delivers corporate training to Fortune 500 audiences in India and the United States and writes regularly on LinkedIn and TheDBAdmin.com on the intersection of databases, infrastructure, and applied AI.

Contact and further reading:

Website: <https://thedbadmin.com>

LinkedIn: [linkedin.com/in/versha-thedbadmin](https://www.linkedin.com/in/versha-thedbadmin)

Email: contact@thedbadmin.com

Thank you for reading.

© 2026 TheDBAdmin.com — All rights reserved.